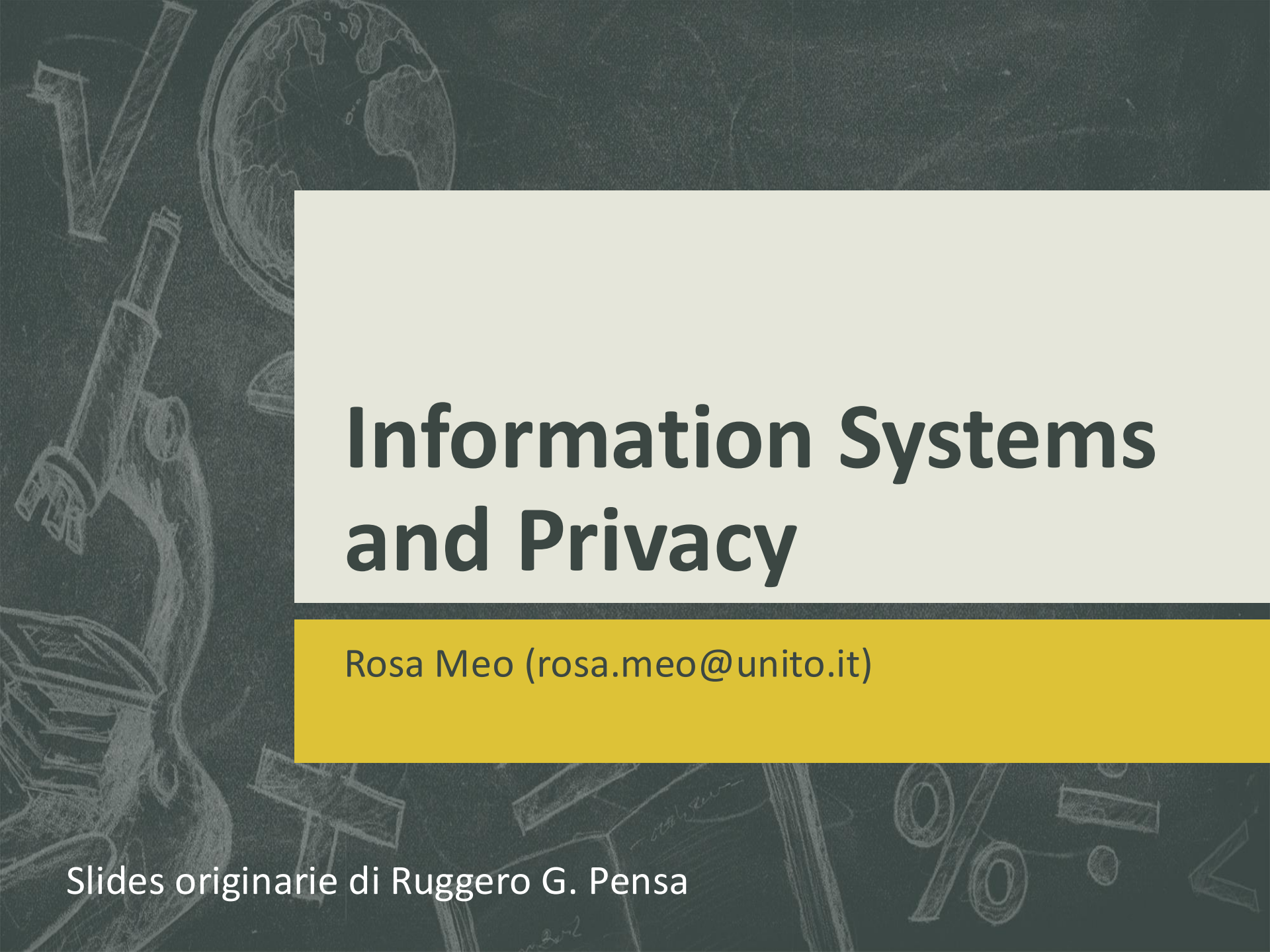


The background is a dark grey chalkboard with various white chalk sketches. On the left, there's a large sketch of a telescope. Above it, a globe is visible. In the bottom right corner, there are sketches of a percentage sign, an exclamation mark, and a less-than sign. The title 'Information Systems and Privacy' is centered in a white box.

Information Systems and Privacy

Rosa Meo (rosa.meo@unito.it)

Slides originarie di Ruggero G. Pensa

Information Systems

- **Information systems** are formal, sociotechnical, organizational systems designed to **collect, process, store,** and **distribute** information used by an organization
- A **computer information system** is a system composed of people and computers that processes or interprets information

Computer Information Systems

A set of software and hardware components that work in concert with one another to:

- collect/acquire
- store
- process
- distribute

information with the goal of supporting:

- the **activities** (automation)
- the **decisions** taken by the governing bodies of an organization
 - analysis
 - control
 - coordination
 - visualization and reporting

Information systems and (private) data

- Information Systems work thanks to the exchange of data/information
 - They can be modeled as a set of interconnected processes where the output of a process is the input of another process
- Data belongs to different entities
 - employees
 - customers/clients/users
 - suppliers
 - ...
- Different types of data exist
 - personal identification data
 - user-generated content
 - e-mails
 - transaction data
 - process log data
 - application data
 - ...

Information Systems and Privacy

- Privacy can be enforced at different levels
 - Educating employees and managers
 - Strengthening cybersecurity measures
 - Protecting networks and server infrastructures
 - Restricting physical access to sensitive areas
 - Using VPN for remote work
 - Keeping software up-to-date
 - ...
- What about Data Protection and GDPR compliance?

Information Systems and the GDPR - Example

Alice is applying for a mortgage for a new home and requests advice from an independent intermediary/advisor  mutuo

The advisor needs to discuss and understand a range of personal information in order to recommend the best available product for Alice's situation

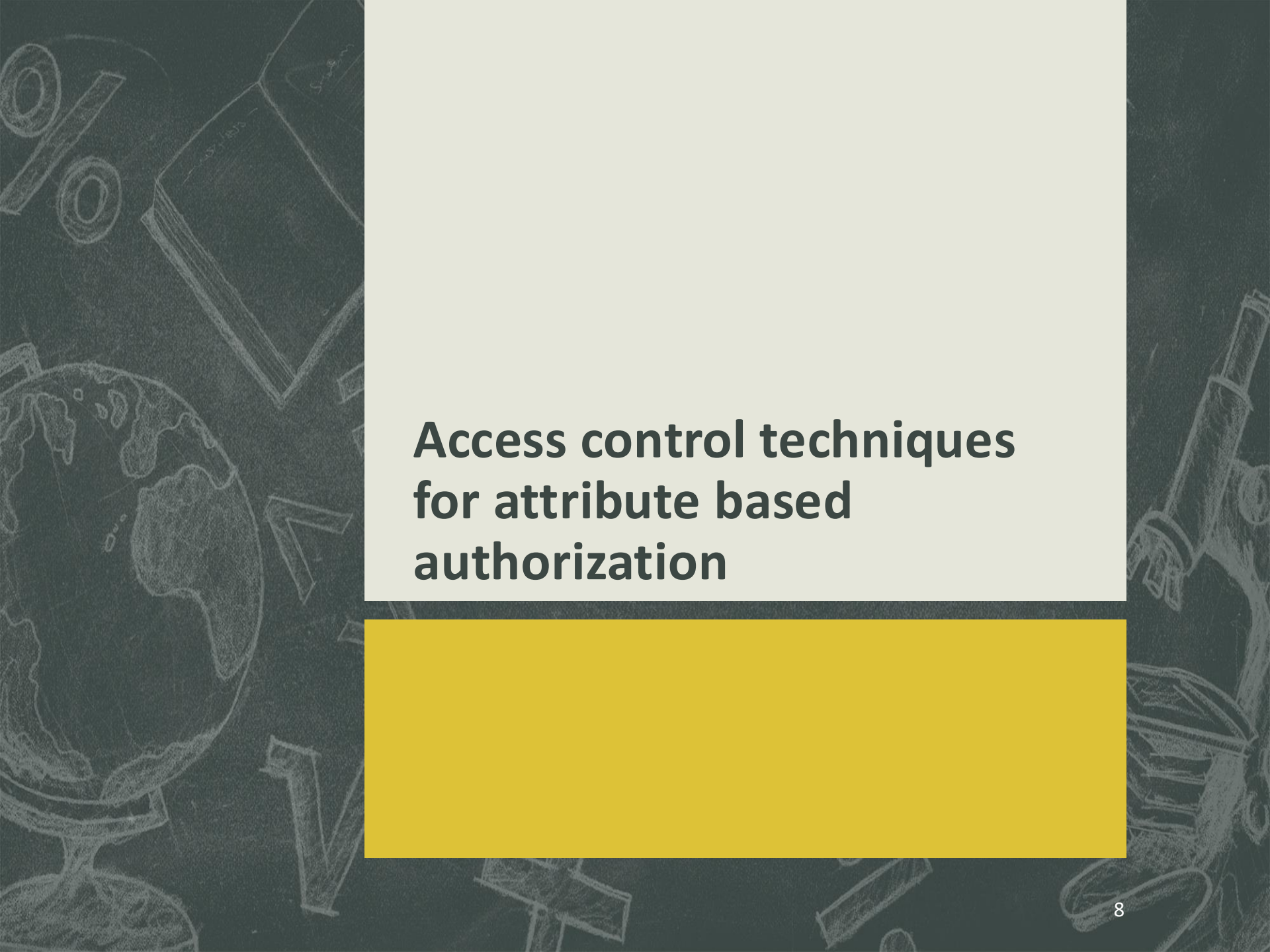
The advisor may request quotations from mortgage providers, based on Alice's circumstances

- Which parties have stored personal information about Alice in this process?
- How can Alice find out where, and by whom this information is stored?
- Do all parties involved meet the demands of the GDPR legislation?
- How can Alice ensure that personal information that parties store about can also be deleted?

Information Systems and the GDPR

A GDPR-compliant information system must ensure:

- **Attribute-based authorization:** a declarative and fully traceable authorization mechanism for accessing all data, under predefined circumstances
- **Anonymizing / pseudonimizing data:** secure mechanisms for the pseudonimization or anonymization of information
- **Traceability:** a register of who has created, modified or deleted information, when and for which purpose
- **Data deletion:** secure mechanisms to ensure the "right to be forgotten" without affecting the reliability of the system



Access control techniques for attribute based authorization

Access control

Access control: restricting access for computer resources, especially in multi-user, data sharing settings

- Privacy and security requirements to be maintained in information systems along with the associated complexity adds to the difficulty in realizing efficient access control schemes
 - E.g., a patient may require her electronic health records to be accessed only by a doctor registered in a specific unit of a specific medical center
 - However, in emergency situations, it is important to give the access to the patient's health records to any doctor
- In this scenario, being a doctor would not be enough to grant the access

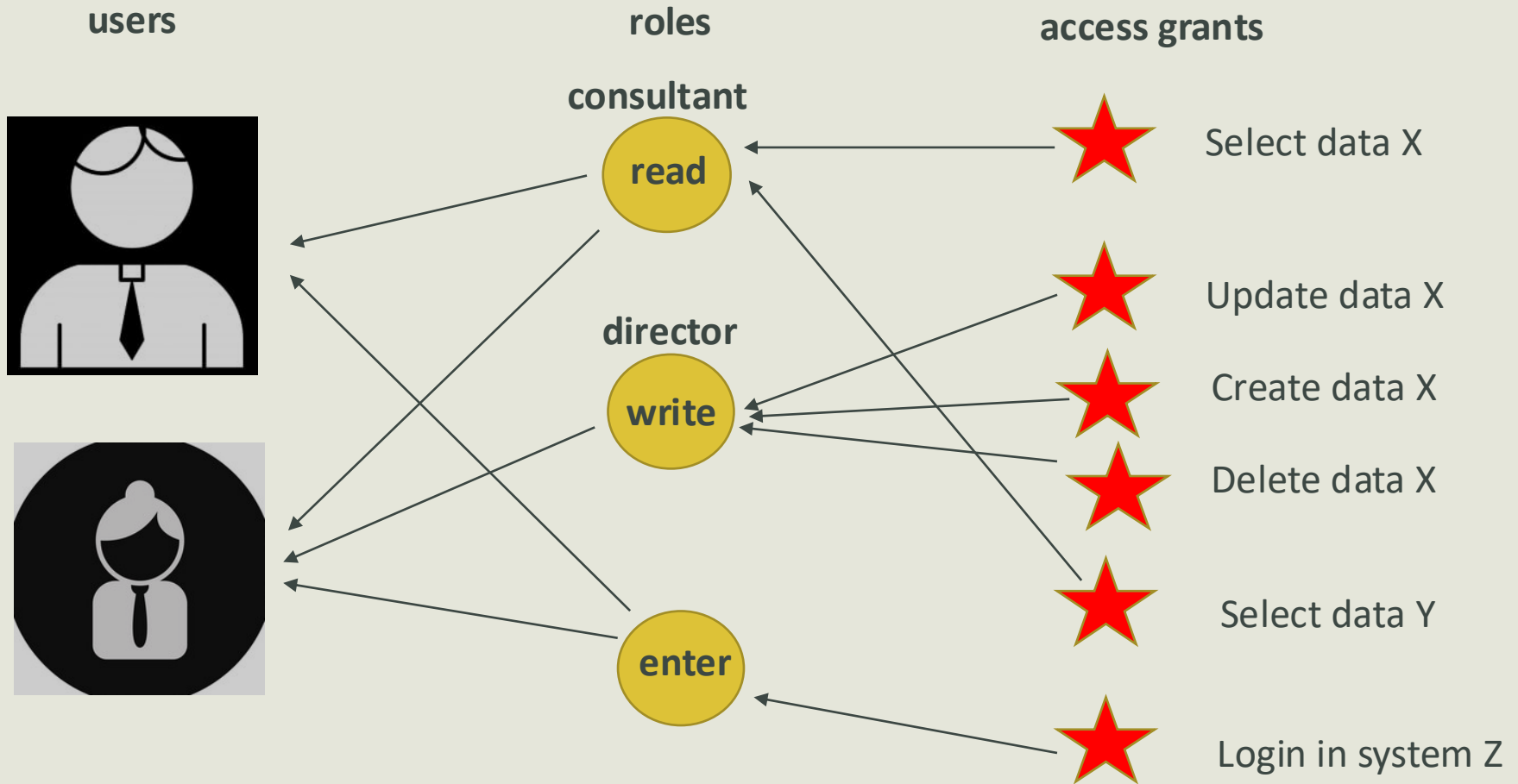
Role based access control (RBAC)

- Formalized by NIST (US National Institute of Standards and Technology) in 1992
- Suitable for enterprises managing upwards of 500 employees
- **Manage access control by role** rather than an employee's user ID
- The role provides an **extra level of abstraction**
 - Acting as a collection of permissions or entitlements
- Each role could be assigned to one or hundreds of users
 - Making management a simpler task

How does RBAC works?

- Administrators **assign access permissions to roles**
- **Roles can be assigned to individual users**
 - Users may have one or several roles (each with different access rights)
- **Administrators** can simply update roles or access permissions by **assigning/removing users** to the **appropriate roles**

Roles, users, grants



RBAC: limitations

- RBAC provides **coarse-grained, predefined** and static access control configurations
- It fails to provide a **flexible** mechanism by which customers/client/users as well as institutions can express their **requirements**
- It also fails to capture the **purpose** for which data will be released to the various stakeholders

RBAC: limitations

- Today, defining authorization policies based solely on a user's role is not good enough
- The context surrounding that user, their data, and the interaction between the two are also important to provide access to
 - the right user
 - at the **right time**
 - in the **right location**
 - and by meeting regulatory compliance
- That means **evolving** an existing Role Based Access Control (RBAC) model to an Attribute Based Access Control (ABAC) model

Access control and context

By **adding context**, authorization decisions can be made based on

- Role of the user
- Who or what that user is **related to**
- What that user needs **access to**
- Where that user needs **access from**
- **When** that user needs access
- How that user is accessing that information

Access control and context

- By defining the context between users and attributes, robust policies can be defined specifying that though a user may create purchase orders, and approve them, he/she cannot approve her/his own purchase orders
- Knowing a user's role is no longer enough to ensure safe and secure access control
- The **context** and the **relationship** information between various entities involved in the system are required
- Attribute Based Access Control allows an enterprise to **extend existing roles using attributes and policies**

Attribute based access control (intuition)

- ABAC uses policies built upon the individual attributes using natural language
- For example, a policy may be written as follows:
“Doctors can view medical records of any patient in their department and update any patient record that is directly assigned to them, during working hours and from an approved device.”
- By creating a policy that is easy to understand, with context around a user and what he/she should have access to, access control becomes far more robust

Attribute based access control

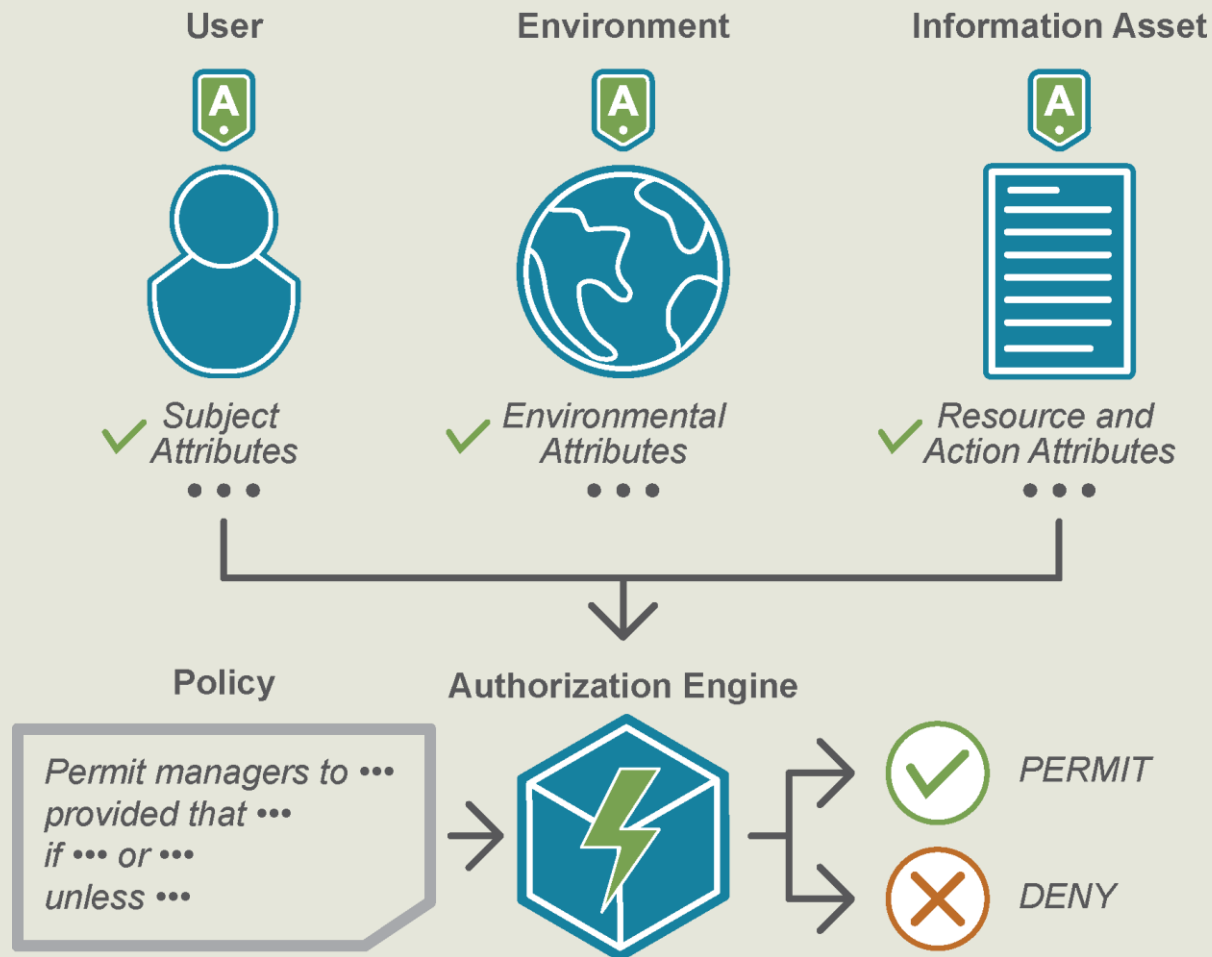
- Similarly to RBAC it adopts a **policy driven approach**
- Uses attributes of **subjects, objects**, and the **environment** (instead of roles)

Attributes are used to express rich policies

- Relieves the need for being registered in the system to be able to access shared resources

The case with RBAC based solutions

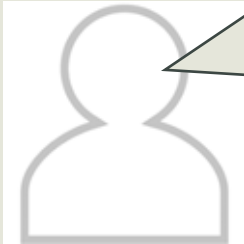
Attribute based access control



Attribute based access control (ABAC)

The **subject** who is demanding access to an asset.

Attributes: roles, group memberships, the user's department / company, management level, etc.



The **action** that the user wants to perform are “read” and/or “write”.

In some situations, the action may be described by a combination of attributes. For example in online banking, you may have “action type= deposit”, and “amount = 1000,-”.



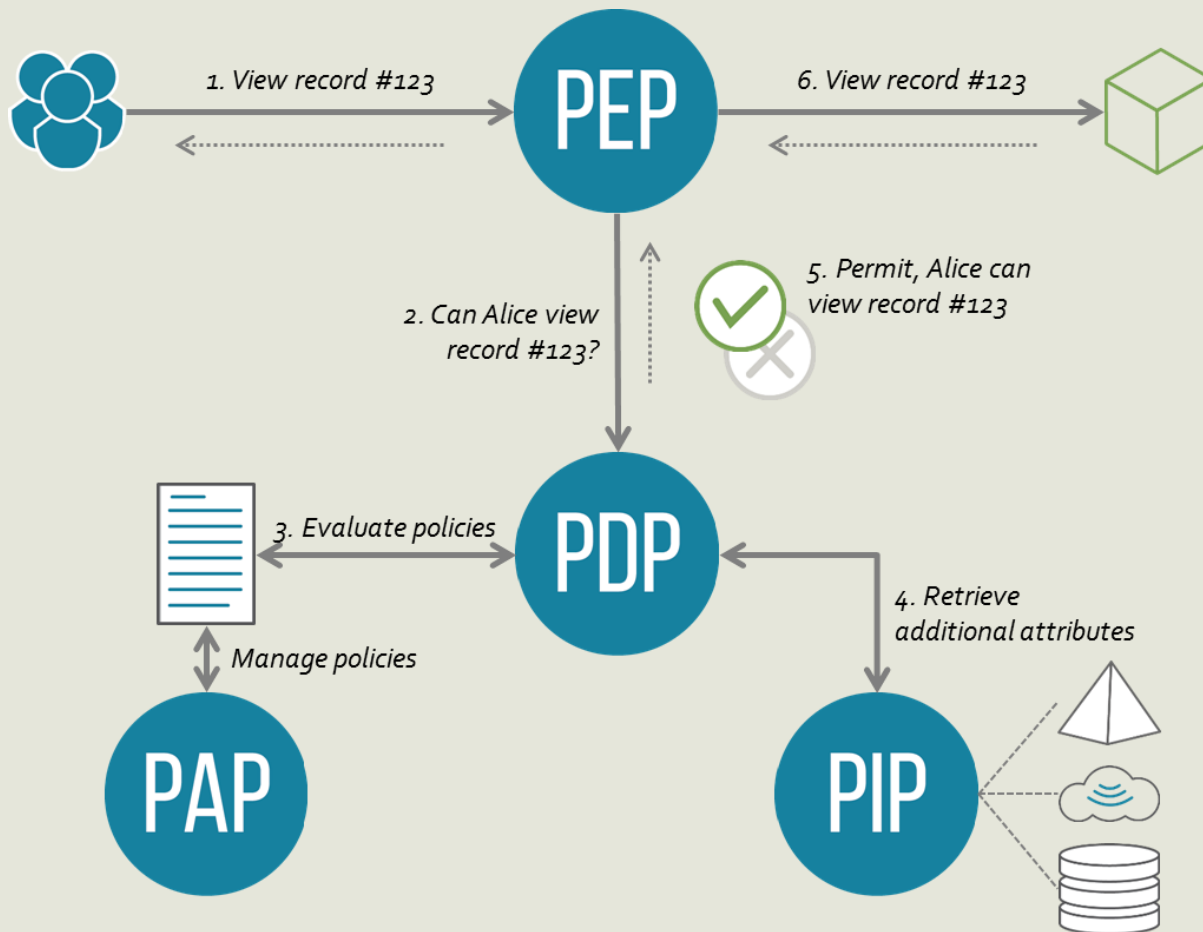
context”.

- A sentence like this includes four grammatical building blocks:
 - a subject
 - an action
 - a resource
 - the environment in which the request is made

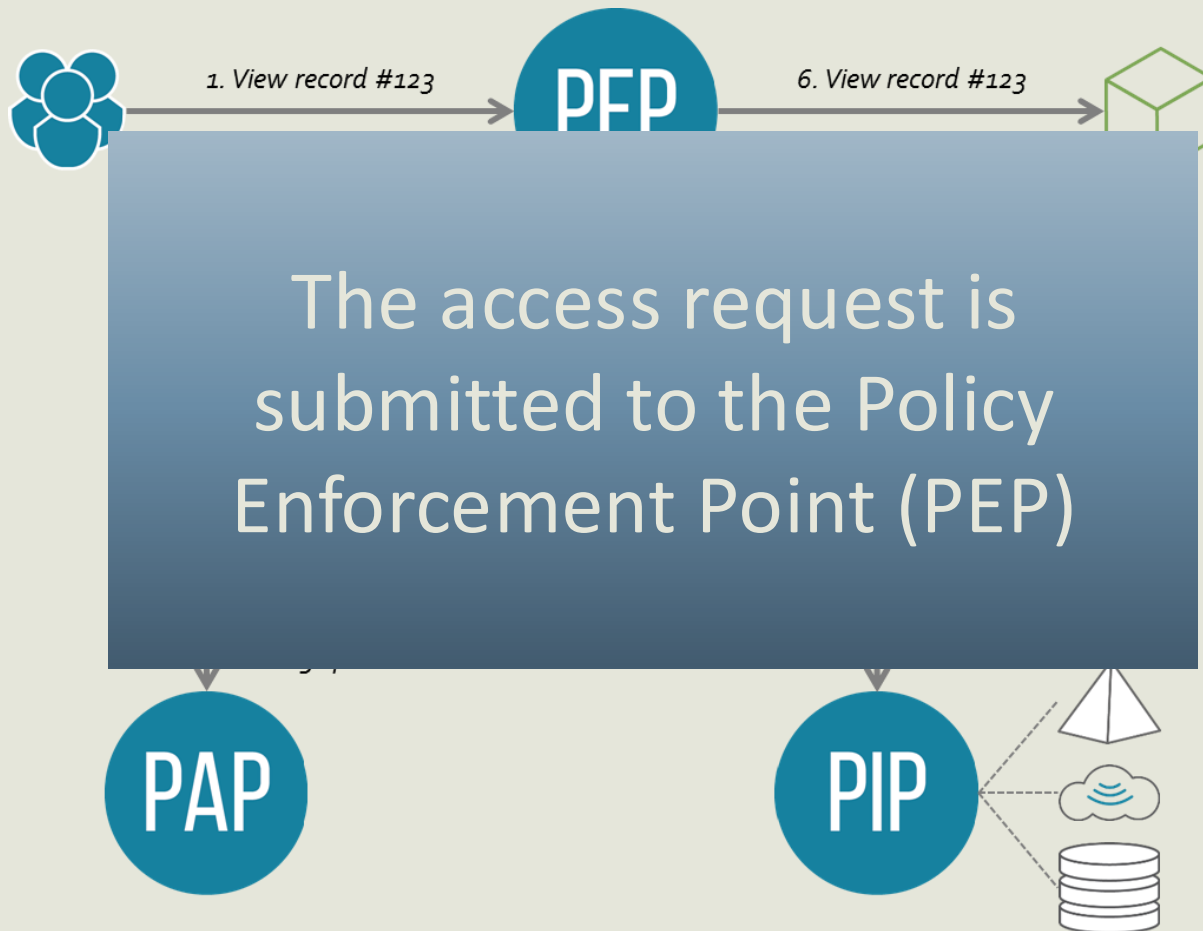
smart phone, etc.).



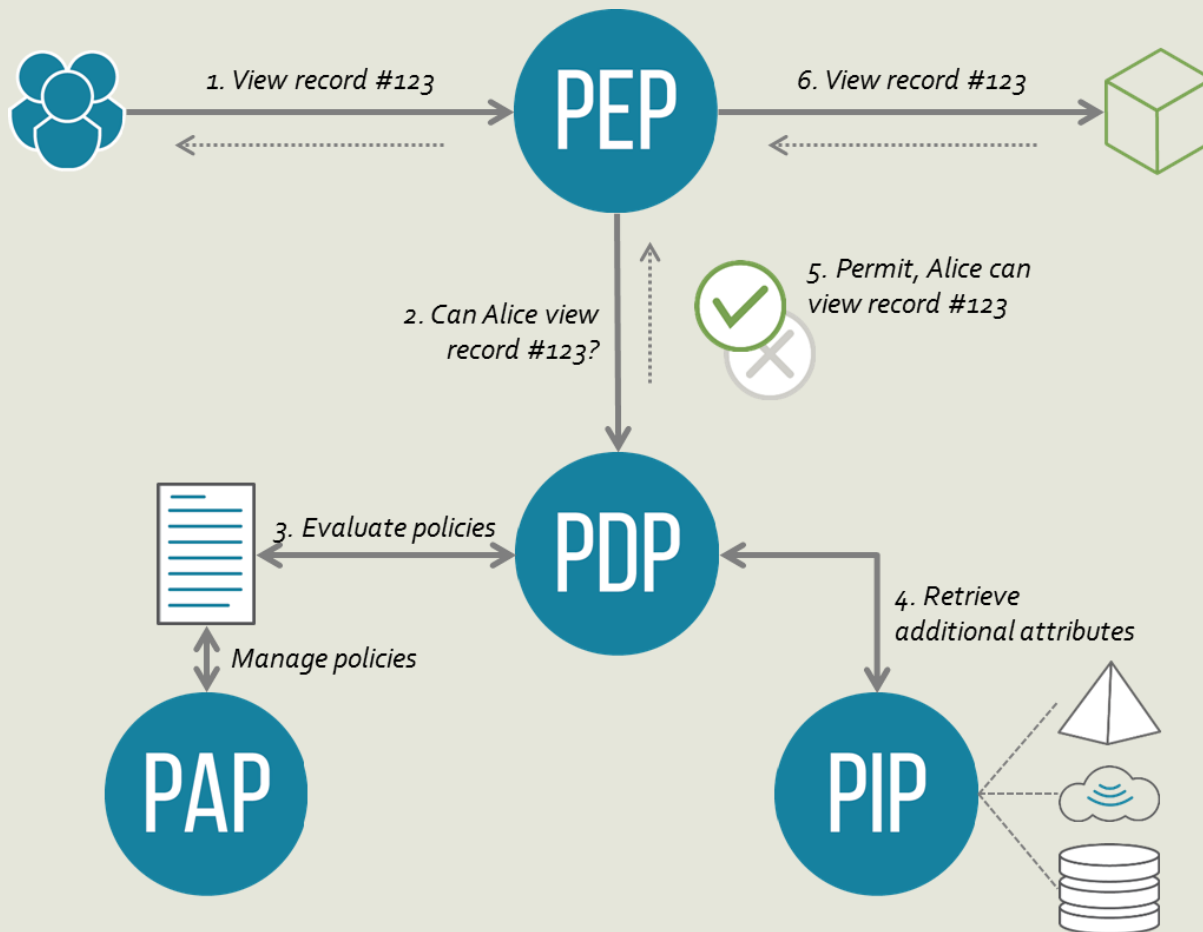
Authorization engine (details)



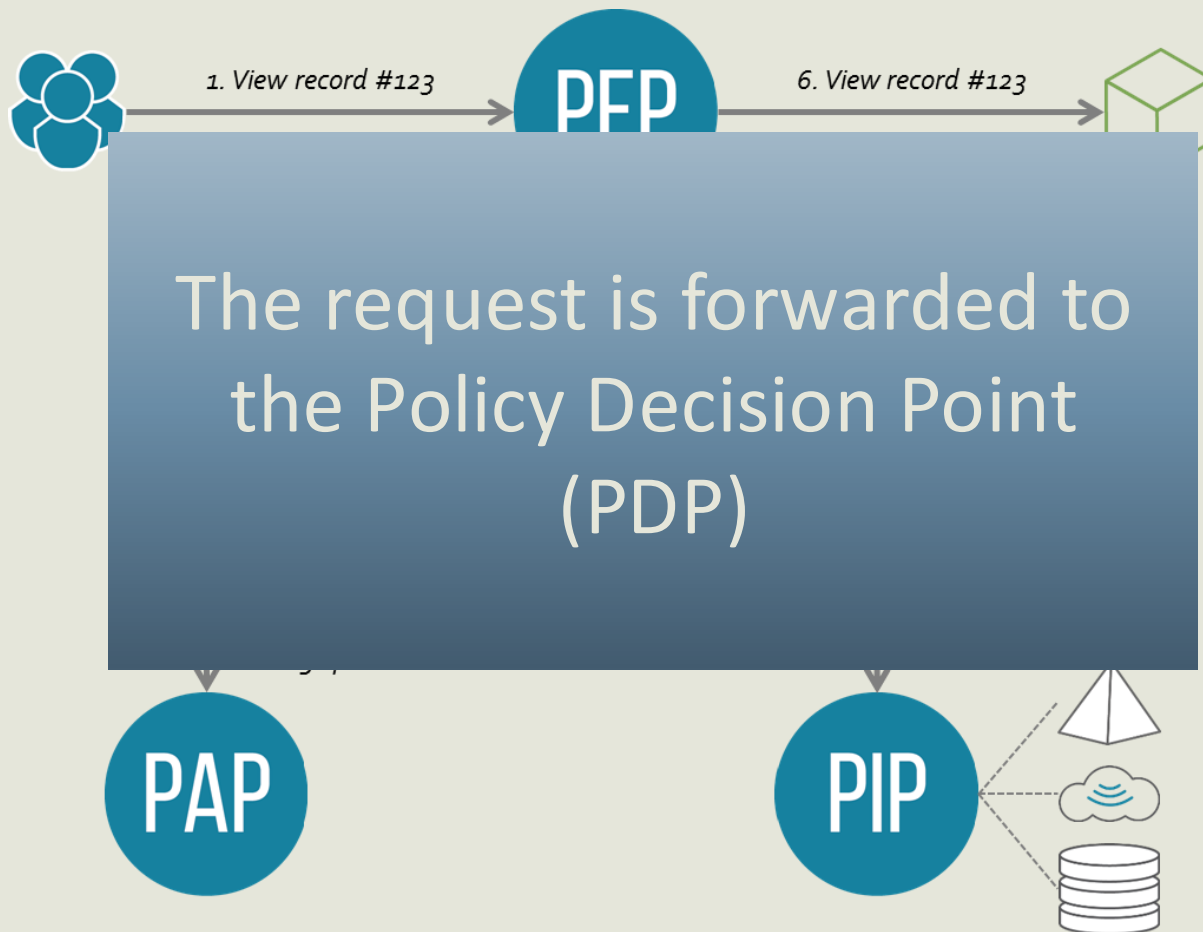
Authorization engine (details)



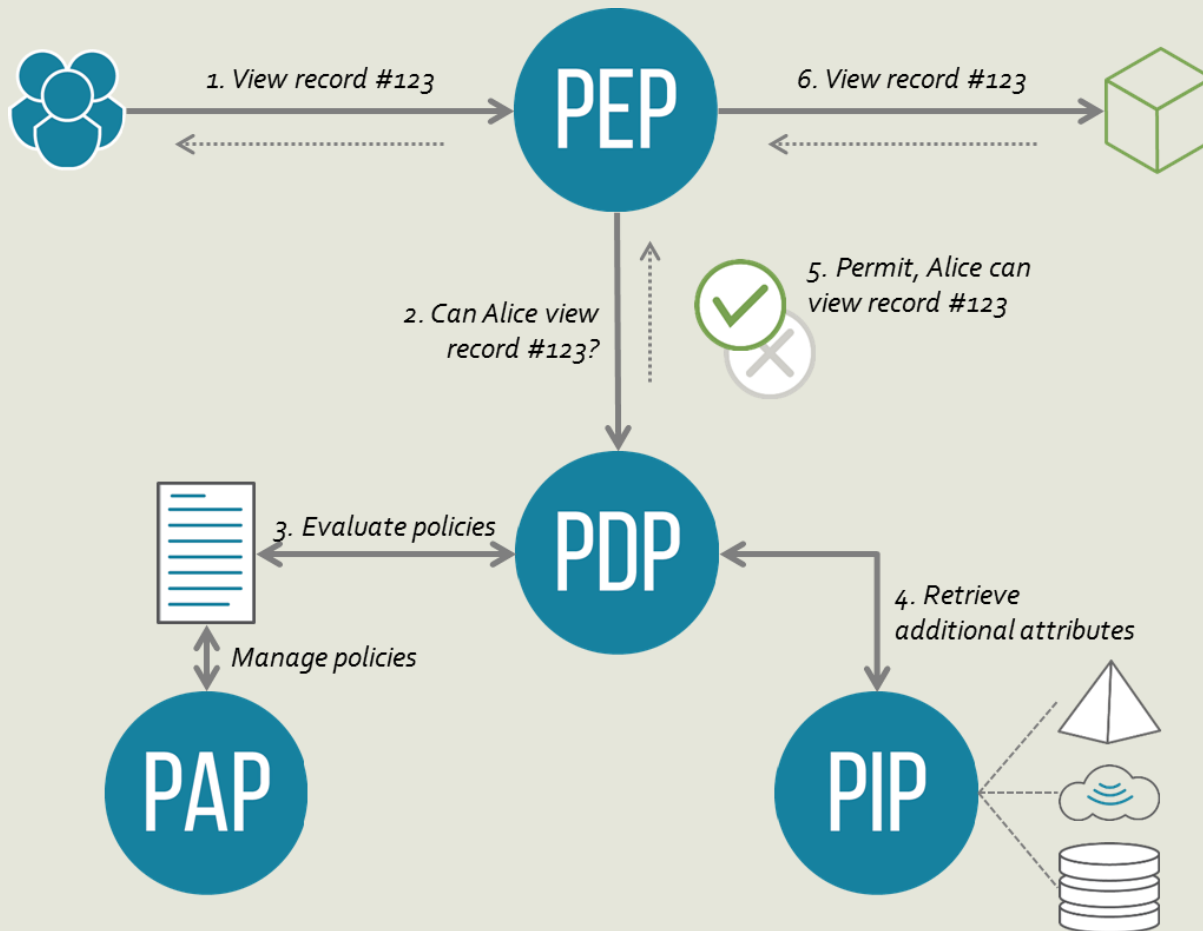
Authorization engine (details)



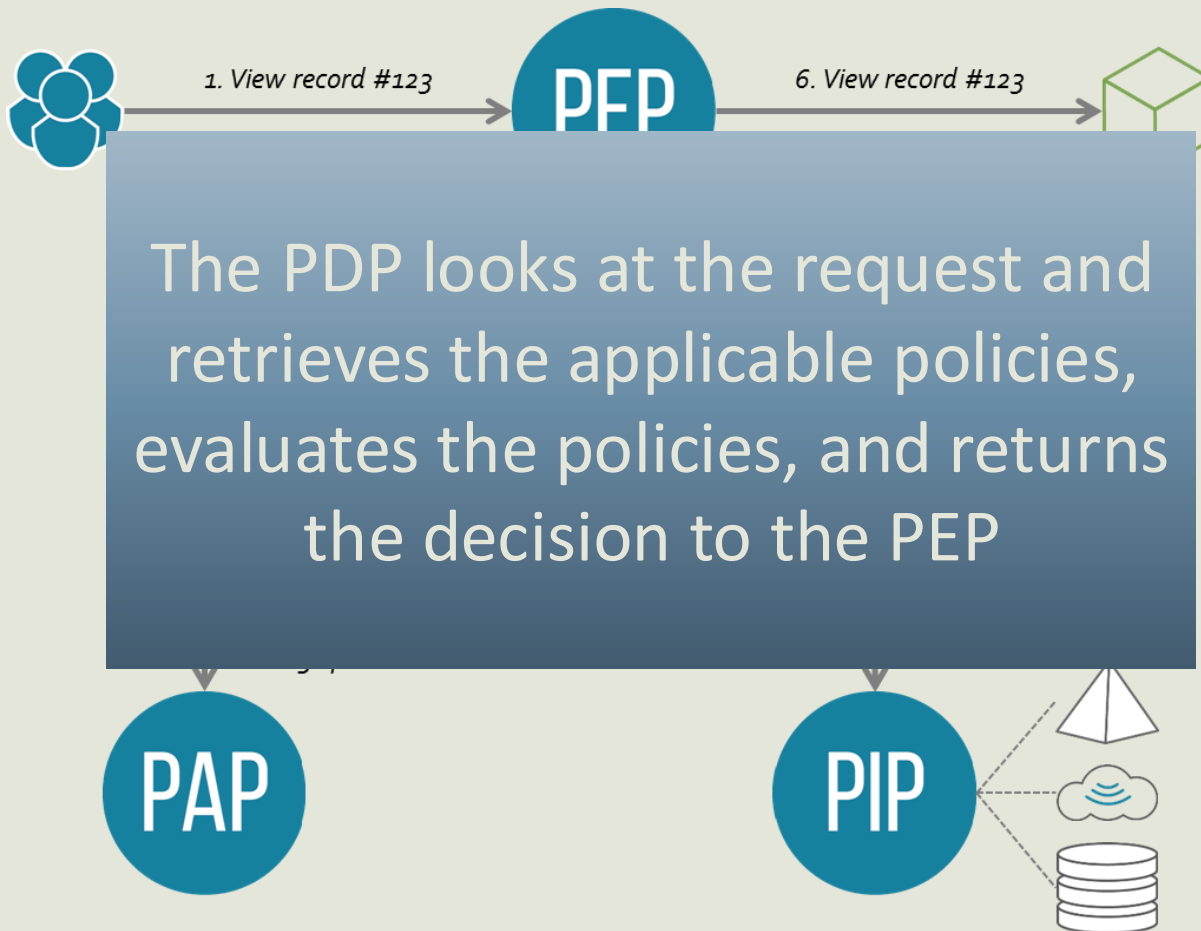
Authorization engine (details)



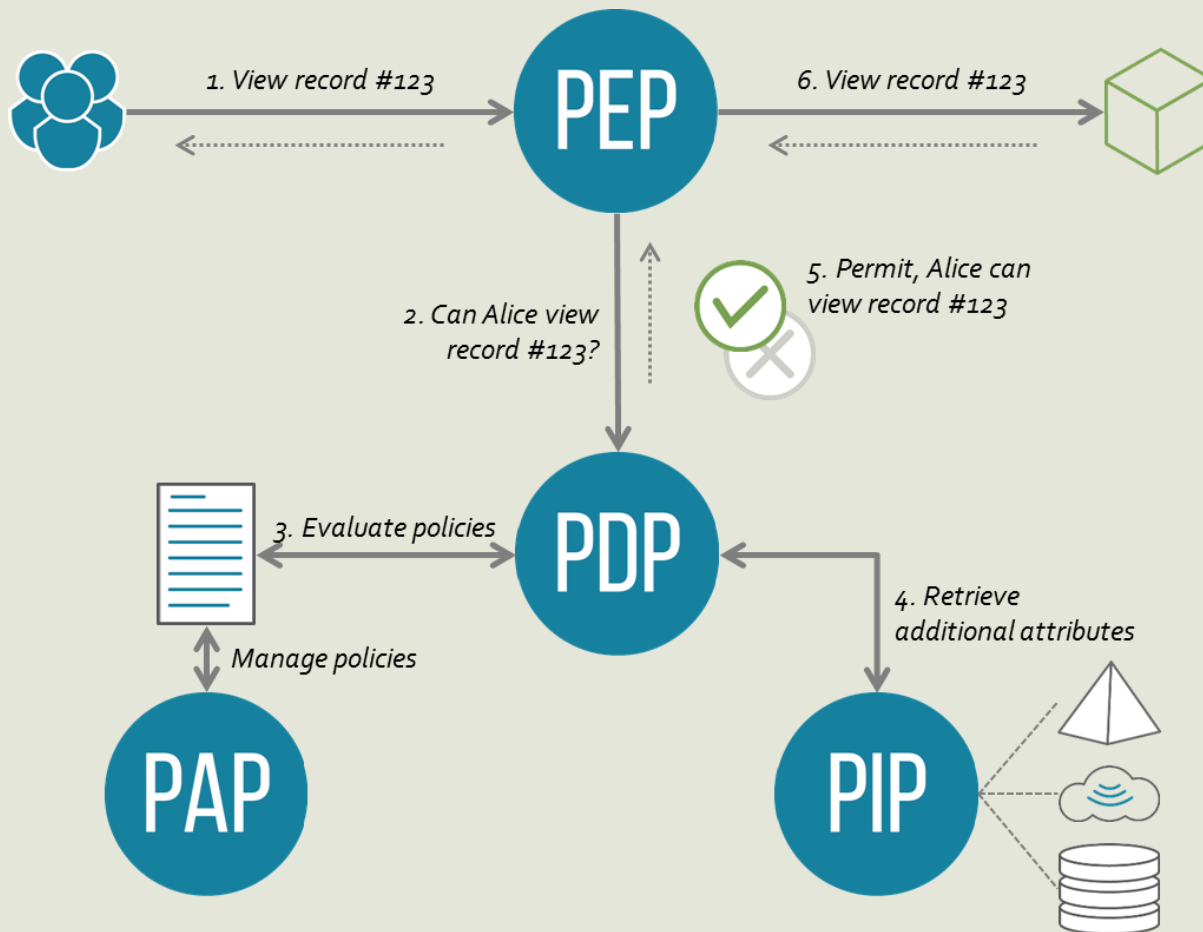
Authorization engine (details)



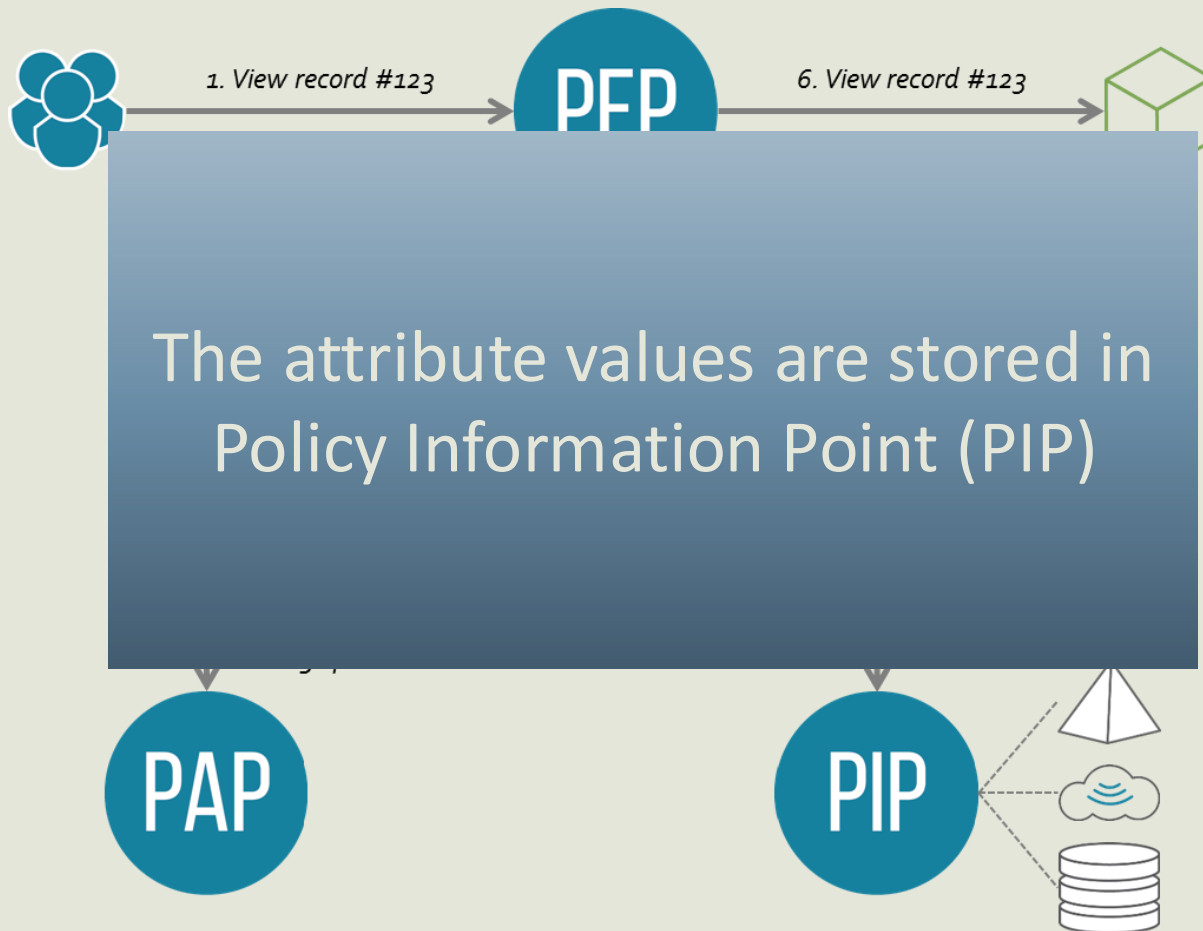
Authorization engine (details)



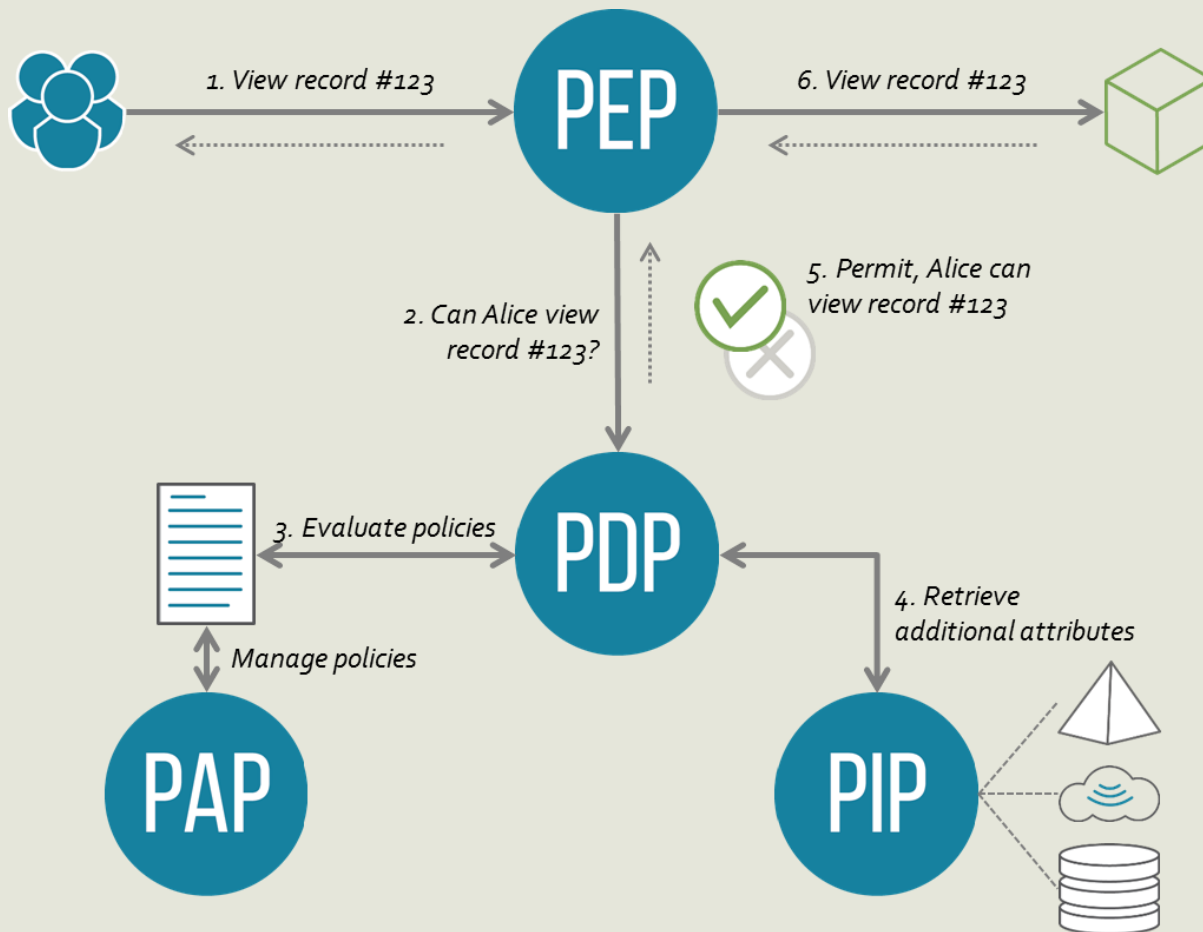
Authorization engine (details)



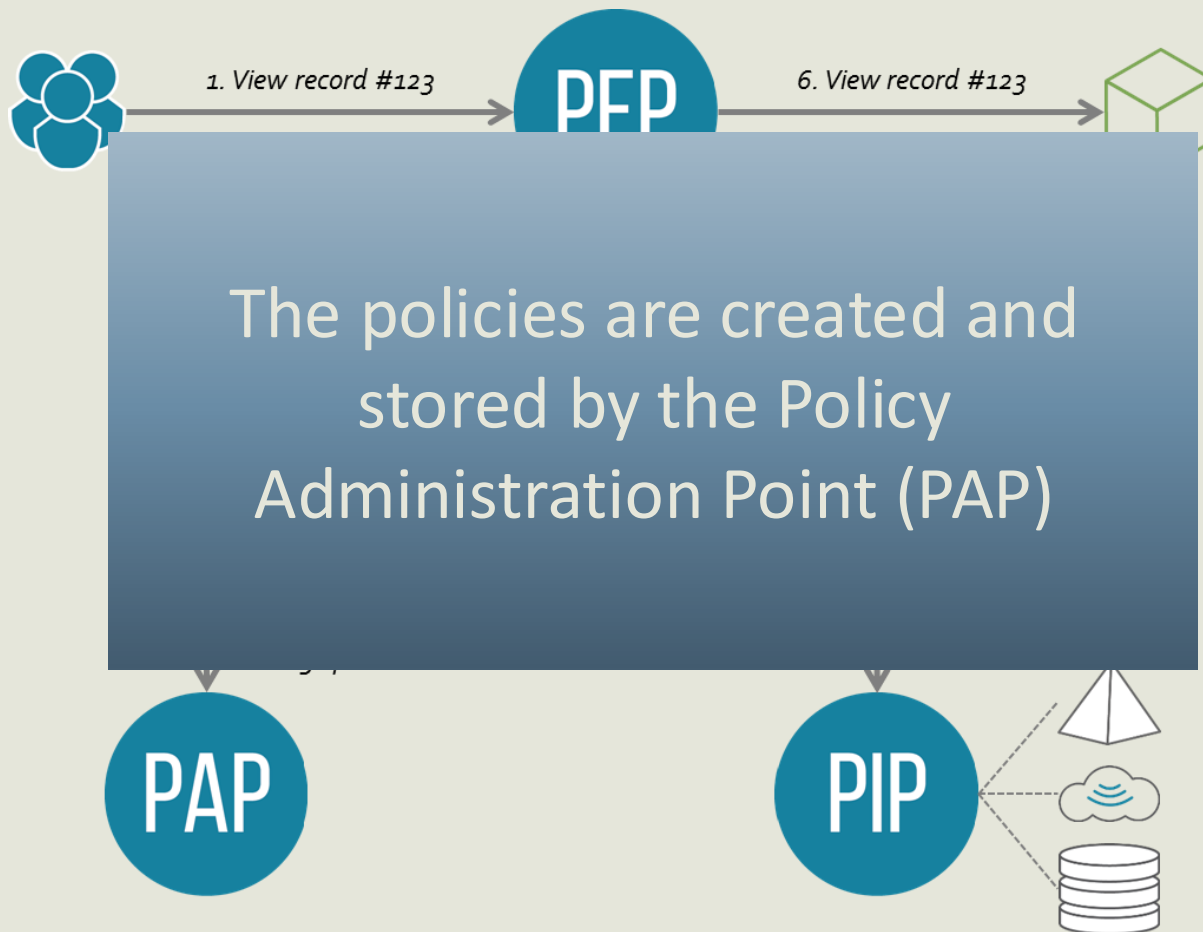
Authorization engine (details)



Authorization engine (details)



Authorization engine (details)



XACML

- The key standard that implements ABAC is XACML
- XACML stands for **eXtensible Access Control Markup Language**
- XML schema for expressing authorization and entitlement policies

Example

- The example is that of a data store that gets clinical health data from various healthcare providers
- Clinical data is received by the data store which must be processed as per the instructions of the patient and the legal rules before it is distributed to authorized researchers
- The example uses ***Institute 1, Institute 2, Patient 1*** and ***Patient 2*** policies for the purpose of demonstration.

Policy of Institute 1

- ***Institute 1*** allows release of patient-level data to **researchers** as limited datasets as long as the requesters can assert **GDPR-compliance**.

Policy of Institute 1 (XACML)

a permit decision has priority over a deny decision

```
<PolicySet PolicySetId = "PolicySetInstitute1" policy-combining-algorithm="permit-overrides">
  <Target>
    /*:Attribute-Category :Attribute ID :Attribute Value */
    AnyOf :access-subject
      :access-subject :Role :Researcher
      :access-subject :Role :Doctor
    AnyOf :resource
      :resource :Type :HealthData
      :resource :Type :AggregateHealthData
    AnyOf :action
      :action :Action-id :Release
      :action :Action-id :Read
      :action :Action-id :Write
  </Target>
  <Policy PolicyId = "Policy1" rule-combining-algorithm="deny-overrides">
    // Institute 1 Rules //
    <Target>
      /*:Attribute-Category :Attribute ID :Attribute Value */
      :access-subject :Role :Researcher
      AnyOf :resource
        :resource :Type :HealthData
        :resource :Type :AggregateHealthData
      :action :Action-id :Release
    </Target>
    <Rule RuleId = "I1R1" Effect="Permit">
      <Condition>
        Function: string-equal
        /*:Attribute-Category :Attribute ID :Attribute Value */
        :access-subject :GDPR Comp :Yes
      </Condition>
    </Rule>
  </Policy>
</PolicySet>
```

Institute 1 allows release to researchers

Institute 1 allows release of patient-level data

a deny decision has priority over a permit decision

Institute 1 allows release to researchers

Institute 1 allows release of patient-level data

Institute 1 allows release to researchers who can assert **GDPR-compliance**.

Policy of Institute 2

- ***Institute 2*** only allows release of **aggregate statistics** (mean, count) and **no** release of patient-level data **to researchers**
- Moreover, this institute **does not allow statistics** to be released that are **calculated** based on **less than 10 patients**
- This institute **does not require** data receivers to be **GDPR compliant**, but **denies** all **individual patient-level data requests**

Policy of Institute 2 (XACML)

```
<PolicySet PolicySetId = "PolicySetInstitute2" policy-combining-algorithm="permit-overrides">
<Target>
/*:Attribute-Category :Attribute ID :Attribute Value */
AnyOf :access-subject
    :access-subject :Role :Researcher
    :access-subject :Role :Doctor
AnyOf :access-resource
    :resource :Type :HealthData
    :resource :Type :AggregateHealthData
AnyOf :action
    :action :Action-id :Release
    :action :Action-id :Read
    :action :Action-id :Write
</Target>
<Policy PolicyId = "Policy2" rule-combining-algorithm="deny-overrides">
<Target>
/*:Attribute-Category :Attribute ID :Attribute Value */
    :access-subject :Role :Researcher
AnyOf :resource
    :resource :Type :AggregateHealthData
    :action :Action-id :Release
</Target>
<Rule RuleId = "I2R1" Effect="Permit">
<Condition>
Function: and
Function: string-equal
/* :Attribute-Category :Attribute ID :Attribute Value */
    :resource :Type :AggregateHealthData
Function: greater than
/* :Attribute-Category :Attribute ID :Attribute Value */
    :resource :PatientCount :10
</Condition>
</Rule>
</Policy>
</PolicySet>
```

to researchers

Institute 2 allows release of
aggregate statistics on
patient-level data

to researchers

Institute 2 allows release of
aggregate statistics

does not allow statistics
based on less than 10 patients

Policies of Patient 1 and 2

- ***Patient 1***, a former drug addict, is **unwilling** to share his/her past drug use information
- ***Patient 2*** is a highly visible politician who **only approves** release of **treatment or visit data** at the **year level granularity**

Policy of Patient 1 (XACML)

```
<PolicySet PolicySetId = "PolicySetPatient1" policy-combining-algorithm="permit-overrides">
<Target>
/* :Attribute-Category :Attribute ID :Attribute Value */
AnyOf :access-subject
      :access-subject      :Role      :Researcher
      :access-subject      :Role      :Doctor
      :resource             :Type      :HealthData
AnyOf :action
      :action               :Action-id  :Release
      :action               :Action-id  :Read
      :action               :Action-id  :Write
</Target>
<Policy PolicyId = "PP1" rule-combining-algorithm="deny-overrides">
// Patient 1 Policy for Researchers
<Target>
/* :Attribute-Category :Attribute ID :Attribute Value */
      :access-subject      :Role      :Researcher
      :resource             :Type      :HealthData
      :action               :Action-id  :Release
      :purpose              :Purpose-id  :DrugEffect
</Target>
<Rule RuleId = "P1R1" Effect="Permit">
<Condition>
Function: and
Function: string-equal
/* :Attribute-Category :Attribute ID :Attribute Value */
      :resource             :Resource-owner :Patient1
Function: string-not-equal
      :resource             :Content       :Drug-usage
</Condition>
</Rule>
</Policy>
</PolicySet>
```

Patient 1 is unwilling to
share drug use information

Policy of Patient 2 (XACML)

```

<PolicySet PolicySetId = "PolicySetPatient2" policy-combining-algorithm="permit-overrides">
  <Target>
    /* :Attribute-Category :Attribute ID :Attribute Value */
    AnyOf :access-subject
      :access-subject :Role :Researcher
      :access-subject :Role :Doctor
      :resource :Type :HealthData
    AnyOf :action
      :action :Action-id :Release
      :action :Action-id :Read
      :action :Action-id :Write
  </Target>
  <Policy PolicyId = "PP2" rule-combining-algorithm="permit-overrides">
    <Target>
      /* :Attribute-Category :Attribute ID :Attribute Value */
      :access-subject :Role :Researcher
      :resource :Type :HealthData
      :action :Action-id :Release
      :purpose :Purpose-id :DrugEffect
    </Target>
    <Rule RuleId = "P2R1" Effect="Permit">
      <Condition>
        Function: and
        Function: string-equal
        /* :Attribute-Category :Attribute ID :Attribute Value */
        :resource :Resource-owner :Patient2
        Function: string-equal
        :resource :Duration :Yearly
      </Condition>
    </Rule>
    <Rule RuleId = "P2R2" Effect="Permit">
      <Condition>
        Function: and
        Function: string-equal
        /* :Attribute-Category :Attribute ID :Attribute Value */
        :resource :Resource-owner :Patient2
        Function: string-equal
        :resource :Duration :Yearly
      </Condition>
    </Rule>
    <Obligations>
      <Obligation Obligation Id="rewriteDate" FullfillOn ="Permit">
        </Obligation>
      </Obligations>
    <Rule RuleId = "P2R3" Effect="Deny">
      </Policy>
    </PolicySet>
  
```

Patient 2 approves release
for the **duration** of the **year**

Rule 2 has an obligation
of **rewriting the date**

Example of access request

- There exists a **researcher** who is requesting the data and he is **GDPR compliant**
- The *researcher* is conducting a study to assess the effectiveness of the experimental drug on hepatitis C symptoms each month in the first year after treatment
- The *researcher* would **need access** to patient's **past drug history** and also **information** about the symptoms at the **month level**

Example of access request (XACML)

```

<Request REQ1>
<Attributes>
    :access-subject      :Subject-id           :Carol
    :access-subject      :Role                 :Researcher
    :access-subject      :GDPR Comp            :Yes
    :resource            :Type                 :HealthData
/* Content */
    :resource            :Content                :Drug-usage
    :resource            :Content                :Symptoms
    :resource            :Content                :Disease
    :resource            :Content                :Duration
    :resource            :Content                :Date
/* Comparing Attribute Values */
    :content-selector :resource.Disease      :HepC
    :content-selector :resource.Duration     :Monthly
    :content-selector :resource.Date        :
    :resource.Date+365>Current Date
    :action            :Action-id            :Release
    :purpose           :Purpose-id           :DrugEffect
</Attributes>
</Request REQ1>

```

a **researcher** is requesting the data and is **GDPR compliant**

patient's **drug data**
symptoms each
month

researcher 's study is on drug on hepatitis C
access to symptoms each month in the first year

Access request processing

- The data store begins by matching the **Target** of the *PolicySetInstitute1*
- The *access-subject* **attribute Role** **matches**
- The *resource* **attribute Type** **matches**
- The *action* attribute **Action-Id** which is **Release** **matches**
- *PolicySetInstitute1* **applies** and **Policy1** must be checked
- The **Target** of **Policy1** **matches**,
 - so the **rule I1R1** is applied
- **I1R1** *has one condition* that checks whether the **access-subject** has the value **Yes** in the **attribute GDPR Comp**
- The condition of **I1R1** is **satisfied** and the effect is **Permit**
 - Since this is the only rule, the net effect of **Policy 1** is **permit**
 - So the effect of **PolicySetInstitute1** is **permit**

Policy of Institute 1 (XACML)

```
<PolicySet PolicySetId = "PolicySetInstitute1" policy-combining-algorithm="permit-overrides">
  <Target>
    /*:Attribute-Category :Attribute ID :Attribute Value */
    AnyOf :access-subject
      :access-subject :Role :Researcher
      :access-subject :Role :Doctor
    AnyOf :resource
      :resource :Type :HealthData
      :resource :Type :AggregateHealthData
    AnyOf :action
      :action :Action-id :Release
      :action :Action-id :Read
      :action :Action-id :Write
  </Target>
```

The *access-subject* attribute **Role** matches

The *resource* attribute **Type** matches

The *action* attribute **Action-Id** matches **Release**

target applies and **Policy1** must be checked

```
<Policy PolicyId = "Policy1" policy-combining-algorithm="deny-overrides">
  // Institute 1 Rules //
```

```
<Target>
  /* :Attribute-Category :Attribute ID :Attribute Value */
  :access-subject :Role :Researcher
  AnyOr :resource
    :resource :Type :HealthData
    :resource :Type :AggregateHealthData
  :action :Action-id :Release
</Target>
```

Target matches (Researcher)

```
<Rule RuleId = "I1R1" Effect="Permit">
```

the rule **I1R1** is applied

```
<Condition>
  Function: string-equal
  /* :Attribute-Category :Attribute ID :Attribute Value */
  :access-subject :GDPR Comp :Yes
</Condition>
</Rule>
</Policy>
</PolicySet>
```

I1R1 condition checks the *access-subject* has the value **Yes** in the attribute **GDPR Comp**

The condition of **I1R1** is **satisfied** and the effect is **Permit**

Access request processing

- The data store then applies the policy set **PolicySetInstitute2** corresponding to **Institute 2**
- The **Target** of **PolicySetInstitute2** is successfully **matched** and *Policy2 is applied*
- The request **fails** to match **Target** of **Policy 2** and so this **policy is inapplicable**
- Thus, **no policy is applicable** for this request from **Institute 2**

Policy of Institute 2 (XACML)

```
<PolicySet PolicySetId = "PolicySetInstitute2" policy-combining-algorithm="permit-overrides">
<Target>
/*:Attribute-Category :Attribute ID :Attribute Value */
AnyOf :access-subject
      :access-subject      :Role      :Researcher
      :access-subject      :Role      :Doctor
AnyOf :access-resource
      :resource            :Type      :HealthData
      :resource            :Type      :AggregateHealthData
AnyOf :action
      :action              :Action-id :Release
      :action              :Action-id :Read
      :action              :Action-id :Write
</Target>
<Policy PolicyId = "Policy2" rule-combining-algorithm="deny-overrides">
<Target>
/*:Attribute-Category :Attribute ID :Attribute Value */
AnyOf :resource
      :resource            :Type      :AggregateHealthData
      :action              :Action-id :Release
</Target>
<Rule RuleId = "I2R1" Effect="Permit">
<Condition>
Function: and
Function: string-equal
/* :Attribute-Category :Attribute ID :Attribute Value */
      :resource            :Type      :AggregateHealthData
Function: greater than
/* :Attribute-Category :Attribute ID :Attribute Value */
      :resource            :PatientCount :10
</Condition>
</Rule>
</Policy>
</PolicySet>
```

Target of PolicySetInstitute2 is matched

The *researcher* needs access

Policy2 is applied

The request **fails** to match Target because the request is access to patient's past drug

This policy is inapplicable

Access request processing

- **PolicySetPatient 1** belonging to **Patient 1** is then considered
- The **Target** of *PolicySetPatient1* **matches** the **attributes** of the *researcher* and his access request so the policy **PP1** is applied
- **Target** of **PP1** **matches** and so rule **P1R1** is applied
- The condition in **P1R1** is *unsatisfied*
 - as the *researcher requests the drug-usage*, so the **rule is not applied**
- Thus, **no policies** of *Patient 1* is able to **satisfy** the request

Policy of Patient 1 (XACML)

```
<PolicySet PolicySetId = "PolicySetPatient1" policy-combining-algorithm="permit-overrides">
<Target>
/* :Attribute-Category :Attribute ID :Attribute Value */
AnyOf :access-subject
      :access-subject      :Role      :Researcher
      :access-subject      :Role      :Doctor
      :resource             :Type      :HealthData
AnyOf :action
      :action               :Action-id  :Release
      :action               :Action-id  :Read
      :action               :Action-id  :Write
</Target>
<Policy PolicyId = "PP1" rule-combining-algorithm="deny-overrides">
// Patient 1 Policy for Researchers
<Target>
/* :Attribute-Category :Attribute ID :Attribute Value */
      :access-subject      :Role      :Researcher
      :resource             :Type      :HealthData
      :action               :Action-id  :Release
      :purpose              :Purpose-id :DrugEffect
</Target>
<Rule RuleId = "P1R1" Effect="Permit">
<Condition>
Function: and
Function: string-equal
/* :Attribute-Category :Attribute ID :Attribute Value */
      :resource             :Resource-owner :Patient1
Function: string-not-equal
      :resource             :Content       :Drug-usage
</Condition>
</Rule>
</Policy>
</PolicySet>
```

The **Target** of **PolicySetPatient1** matches

policy PP1 is applied

Target of PP1 matches and so rule P1R1 is applied

The condition in P1R1 is **unsatisfied**

the rule is not applied

as the researcher requests the drug-usage

No policies of Patient 1 satisfy the request

Access request processing

- *PolicySetPatient2* is then considered
- The **Target** of *PolicySetPatient2* **matches** and so the **policy PP2** is **applied**
- The **Target** of **PP2** **matches** and so the **rules P2R1, P2R2** and **P2R3** are **checked**
- The condition of **P2R1** **matches** and the effect is **Permit**
- The condition of **P2R2** **matches** and the effect is **Permit**
- The policy set has an **obligation** that requires the response be rewritten to transform the dates
- **P2R3** is the **default** rule and its effect is **Deny**
- **P2R2** and **P2R3** provide **contradictory** effects,
 - but the *rule-combining algorithm* of **PP2** says that **permit-overrides**
- *Researcher* will **have access** to the date **rewritten** records of Patient 2 that are found in *Institute 1*

Policy of Patient 2 (XACML)

```

<PolicySet PolicySetId = "PolicySetPatient2" policy-combining-algorithm="permit-overrides">
  <Target>
    /* :Attribute-Category :Attribute ID :Attribute Value */
    AnyOf :access-subject
      :access-subject :Role :Researcher
      :access-subject :Role :Doctor
      :resource :Type :HealthData
    AnyOf :action
      :action :Action-id :Release
      :action :Action-id :Read
      :action :Action-id :Write
  </Target>
  <Policy PolicyId = "PP2" rule-combining-algorithm="permit-overrides">
    <Target>
      /* :Attribute-Category :Attribute ID :Attribute Value */
      :access-subject :Role :Researcher
      :resource :Type :HealthData
      :action :Action-id :Release
      :purpose :Purpose-id :DrugEffect
    </Target>
    <Rule RuleId = "P2R1" Effect="Permit">
      <Condition>
        Function: and
        Function: string-equal
        /* :Attribute-Category :Attribute ID :Attribute Value */
        :resource :Resource-owner :Patient2
        Function: string-equal
        :resource :Duration :Yearly
      </Condition>
    </Rule>
    <Rule RuleId = "P2R2" Effect="Permit">
      <Condition>
        Function: and
        Function: string-equal
        /* :Attribute-Category :Attribute ID :Attribute Value */
        :resource :Resource-owner :Patient2
        Function: string-equal
        :resource :Duration :Yearly
      </Condition>
    </Rule>
    <Obligations>
      <Obligation Obligation Id="rewriteDate" FullfillOn ="Permit">
      </Obligation>
    </Obligations>
    <Rule RuleId = "P2R3" Effect="Deny">
    </Rule>
  </Policy>
</PolicySet>

```

The **Target** of **PolicySetPatient2** matches

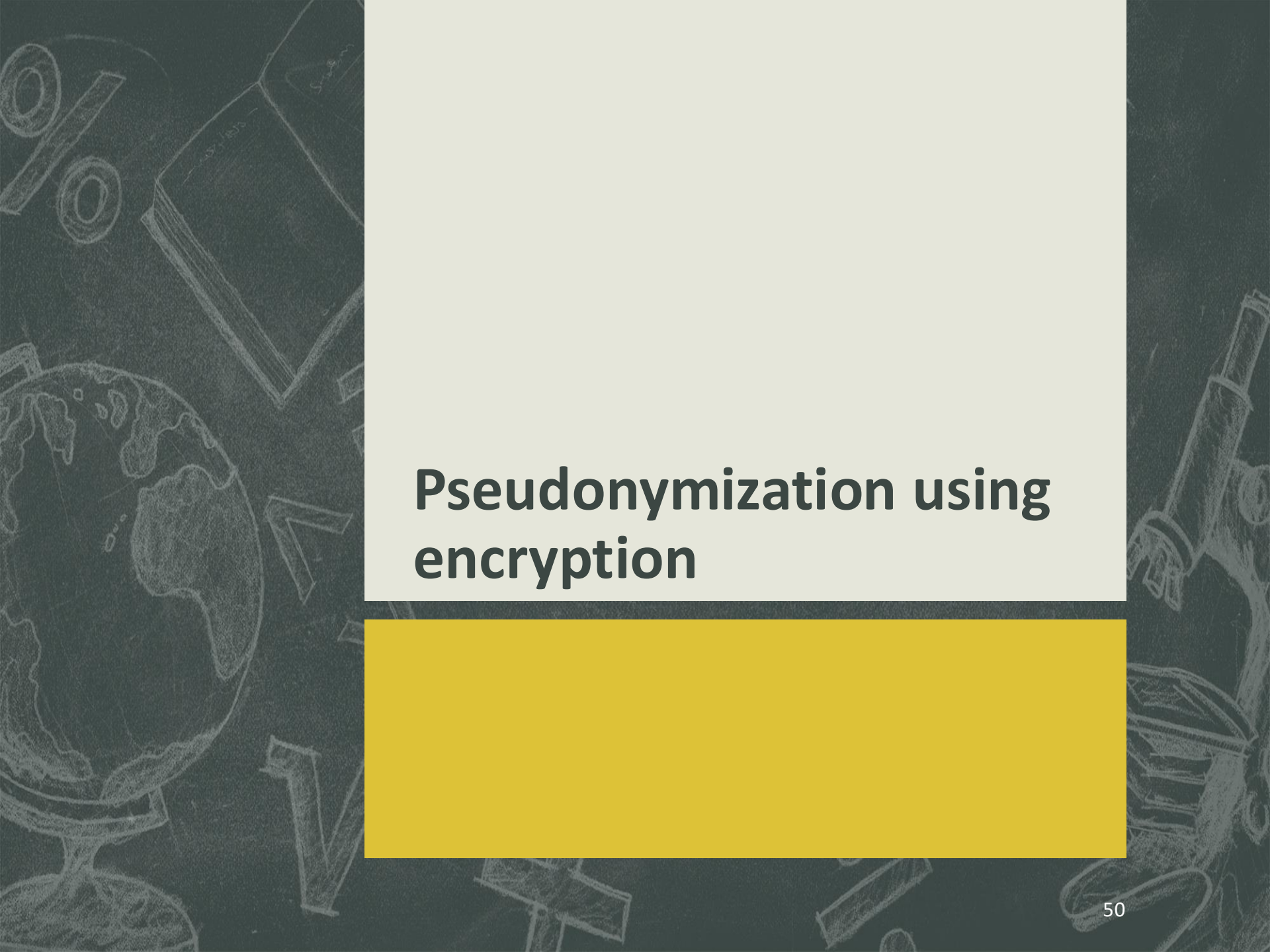
policy PP2 is applied

The **Target** of **PP2** matches: rules **P2R1**, **P2R2**, **P2R3** are checked

P2R1 condition matches: it **permits**

P2R2 condition matches: it **permits**

Conflicting results: combining algorithm is **permit overrides**: request is permitted



Pseudonymization using encryption

Pseudonymization

- **Pseudonymization** is a data management and de-identification procedure by which personally identifiable information fields within a data record are replaced by one or more artificial identifiers, or pseudonyms
- A single pseudonym for each replaced field or collection of replaced fields makes the data record less identifiable while remaining suitable for data analysis and data processing
- Pseudonymized data **can be restored** to its original state with **the addition of information** which then allows individuals to be re-identified
- Anonymized data **can never be restored** to its original state

Column encryption

- **Column encryption** is a feature designed to protect sensitive data stored in databases
- It allows clients to encrypt sensitive data inside client applications and never reveal the encryption keys to the Database Engine
- As a result, encryption provides a separation between those who own the data (and can view it) and those who manage the data (but should have no access)

Column encryption

- When setting up encryption for a column, information about the encryption algorithm and cryptographic keys used to protect the data in the column can be specified
- Two types of keys exist:
 - **column encryption keys**, used to encrypt data in an encrypted column
 - **column master keys**, that encrypt one or more column encryption keys
- The DBMS stores encryption configuration for each column in database metadata (keys are never stored or used in plaintext)
- The DBMS only stores encrypted values of column encryption keys and the information about the location of column master keys, which are stored in external trusted key stores

Deterministic vs. Randomized encryption

The DBMS supports some queries on encrypted data, depending on the encryption type for the column

- **Deterministic encryption** always generates the same encrypted value for any given plain text value, allowing:
 - point lookups
 - equality joins
 - Grouping
 - Indexing
- It may allow unauthorized users to guess information about encrypted values by examining patterns in the encrypted column, especially if there is a small set of possible encrypted values, such as True/False, or North/South/East/West region
- **Randomized encryption** uses a method that encrypts data in a less predictable manner. Randomized encryption is more secure, but prevents searching, grouping, indexing, and joining on encrypted columns

Example (Always Encrypted - MSSQL)

```
CREATE COLUMN MASTER KEY MyCMK
WITH (
    KEY_STORE_PROVIDER_NAME = 'MSSQL_CERTIFICATE_STORE',
    KEY_PATH = 'path'
);
```

```
CREATE COLUMN ENCRYPTION KEY MyCEK
WITH VALUES
(
    COLUMN_MASTER_KEY = MyCMK,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = encryptedkeyvalue
);
```

```
CREATE TABLE Customers (
    CustName varchar(60)
        ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = MyCEK,
            ENCRYPTION_TYPE = RANDOMIZED,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'),
    SSN varchar(11)
        ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = MyCEK,
            ENCRYPTION_TYPE = DETERMINISTIC,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'),
    Age int NULL
);
```

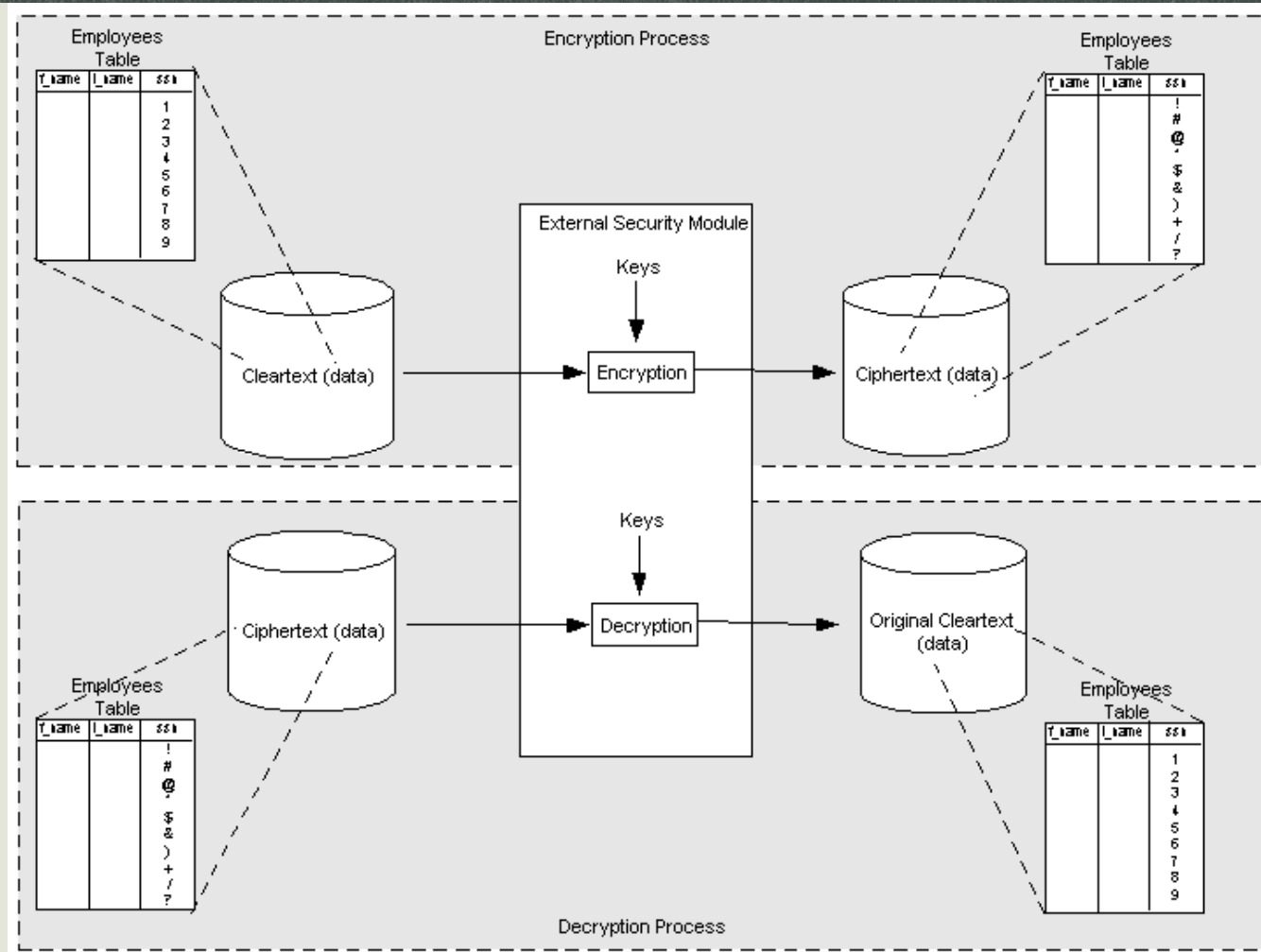
Optimal Asymmetric Encryption Padding (OAEP)
is a padding scheme often used together with RSA encryption.

RSA (Rivest–Shamir–Adleman) cryptosystem

- RSA (Rivest–Shamir–Adleman) is a **public-key** cryptosystem
- It is one of the oldest methods
- RSA user creates and publishes a public key based on two large prime numbers, along with an auxiliary value.
- The **prime numbers are kept secret**.
- Messages can be encrypted by anyone, via the public key
- Messages can only be **decrypted** by someone who knows the private key.
- **Padding** is any of a number of distinct practices which all include adding data to the beginning, middle, or end of a message prior to encryption.
- padding may include **adding nonsense phrases** to a message to obscure the fact that many messages end in predictable ways.

Example:

Transparent Data Encryption (TDE) – Oracle



Generation and storage of the encryption key in TDE (Oracle)

A **single encryption key** is used regardless of the number of encrypted columns.

The keys for all encrypted columns are in turn encrypted with the database server **master key** and stored in a database dictionary table.

No keys are stored in the clear.

The master key of the server is stored in an external security module, outside the database and accessible only to the security administrator.

For this external security module, Oracle uses an **Oracle wallet**.

Storing the master key in this way prevents its unauthorized use.

In addition to storing the master key, the **Oracle wallet is also used to generate encryption keys** and perform encryption and decryption.

Using an external security module separates ordinary program functions from encryption operations, making it possible to **divide duties between database administrators and security administrators**. Security is enhanced because **no single administrator is granted complete access to all data**.

Example: setting the master key in TDE (Oracle)

```
ALTER SYSTEM SET ENCRYPTION KEY "j231m781098dhb345sm"  
IDENTIFIED BY "p3812dH9E";  
  
CREATE TABLE employee (  
    first_name VARCHAR2(128),  
    last_name VARCHAR2(128),  
    empID NUMBER ENCRYPT NO SALT,  
    salary NUMBER(6) ENCRYPT USING '3DES168'  
);
```

certificate_ID

password

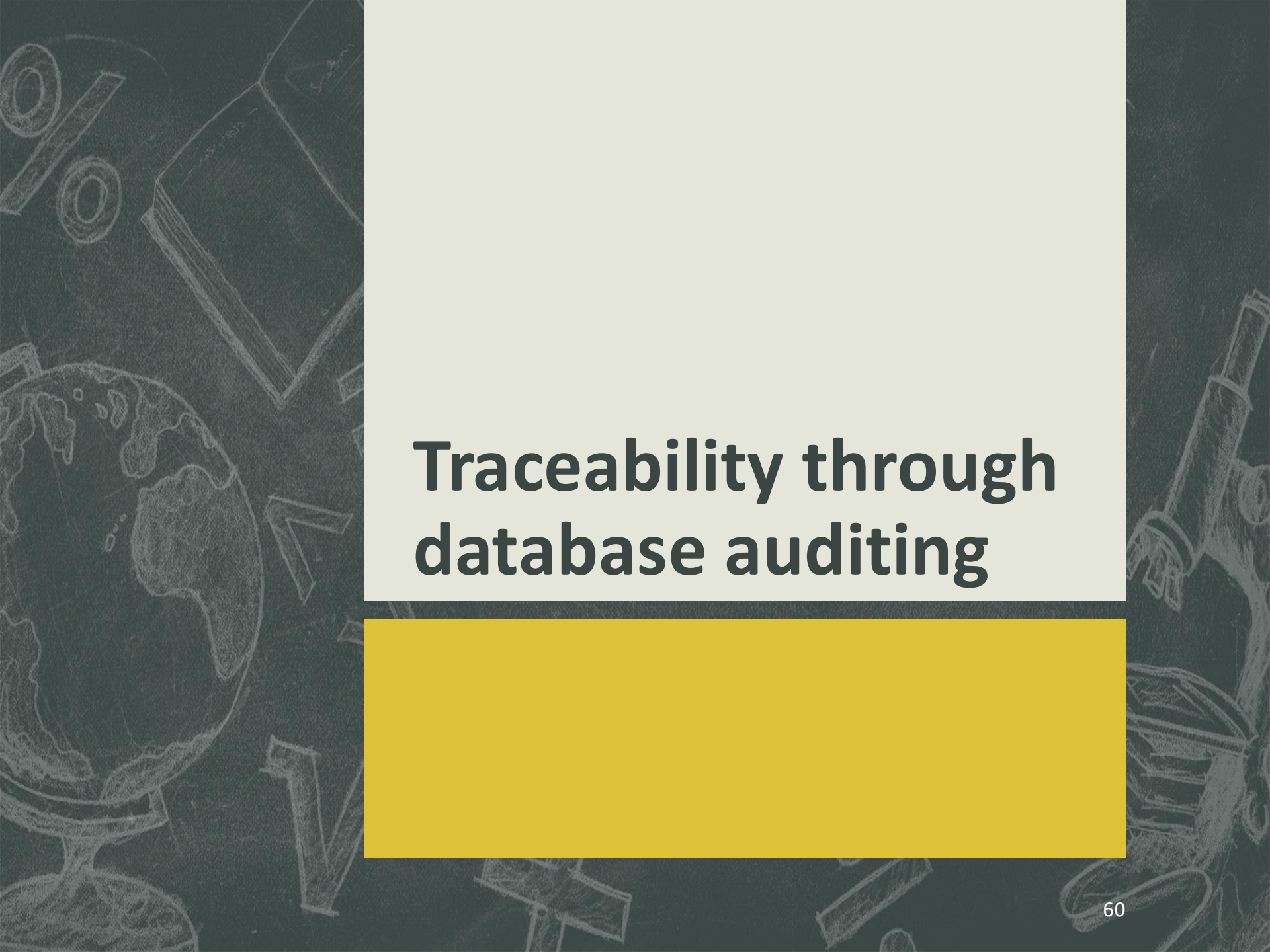
Adding salt, makes brute force data stealing more difficult.
if you index the encrypted column, then you must use NO SALT

Encryption algorithm

Salt is a way to **strengthen the security of encrypted data**.

It is a **random string added to the data before it is encrypted**, causing repetition of text in the clear to appear different when encrypted.

Salt thus removes one method attackers use to steal data, namely, matching patterns of encrypted text.

The background of the slide is a dark grey textured surface. On the left side, there is a faint, light-colored illustration of a globe showing continents. Above the globe, there are sketches of books and a large percentage symbol (%). On the right side, there is a faint illustration of a microscope. The central part of the slide is a white rectangular area containing the title text. Below the white area is a solid yellow rectangular block.

Traceability through database auditing

Auditing – general concepts

- Audit/auditing: **process of examining and validating** documents, data, processes, procedures, systems
- Audit log: **document** that contains all activities that are being audited ordered in a chronological manner
- Audit objectives: set of **business rules, system controls, government regulations, or security policies**

Definitions

- Auditor: a **person** authorized to audit
- Audit procedure: set of **instructions** for the auditing process
- Audit report: a **document** that contains the audit findings
- Audit trail: chronological record of **document changes, data changes, system activities, or operational events**

Types of audits

- Internal auditing
 - examination of activities conducted by staff members of the audited organization
- External auditing
 - examination of the activities of the organization conducted by an independent third party

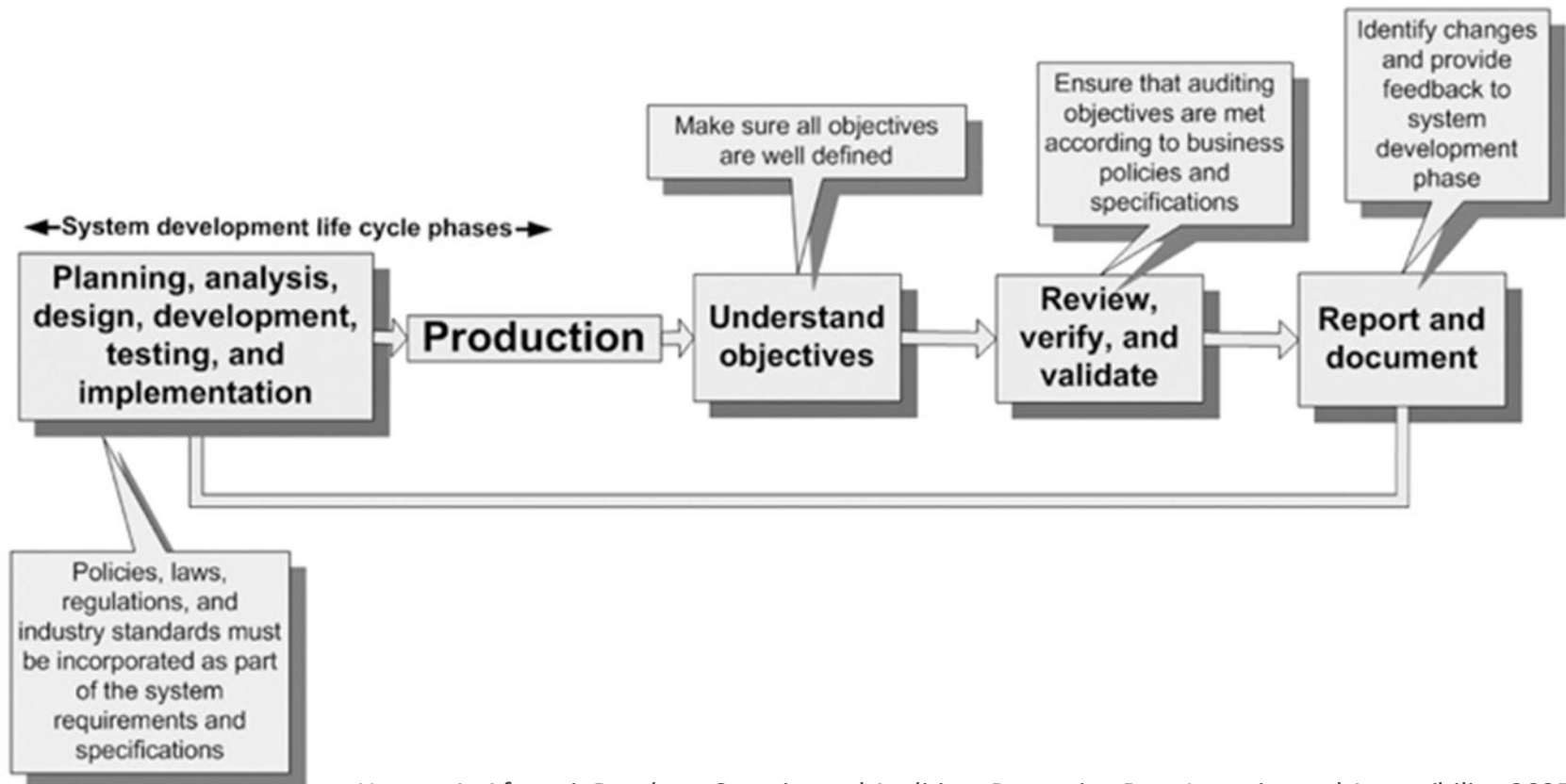
Auditing activities

- Identify security issues that must be addressed
- Establish plans, policies, and procedures for conducting audits
- Organize and conduct internal audits
- Ensure all contractual items are met by the organization being audited
- Act as liaison between the company and the external audit team
- Provide consultation to the Legal Department

Auditing process

- **Auditing process:** ensures that the system is working and complies with the policies, regulations and laws
- Auditing process is done after the product is commissioned into production.
- Auditing process flow:
 - System development life cycle
 - Auditing process:
 - Understand the objectives
 - Review, verify, and validate the system
 - Document the results

Auditing process



Hassan A. Afyouni: Database Security and Auditing: Protecting Data Integrity and Accessibility, 2005

Data and auditing

- **Data audit:** chronological record of data changes stored in log file or database table object
- **Database auditing:** chronological record of database activities

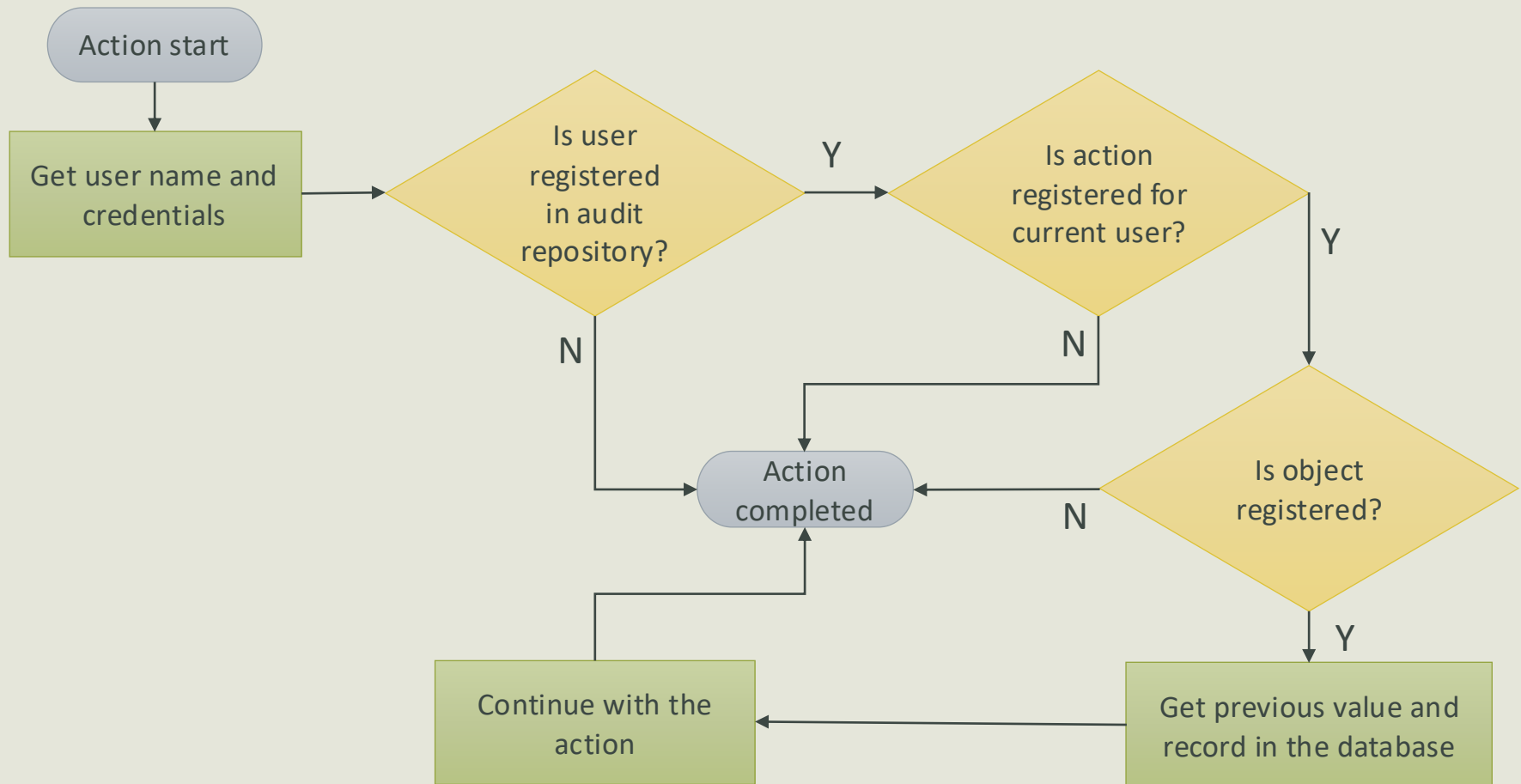
Auditing objectives

- Database auditing objectives:
 - Data integrity
 - Application users and roles
 - Data confidentiality
 - Access control
 - Data changes
 - Data structure changes
 - Database or application availability
 - Change control
 - Auditing reports

Auditing models

- Can be implemented with built-in features or your own mechanism
- Information recorded:
 - **State of the object before** the action was taken
 - Description of the **action** that was performed
 - **Name of the user** who performed the action

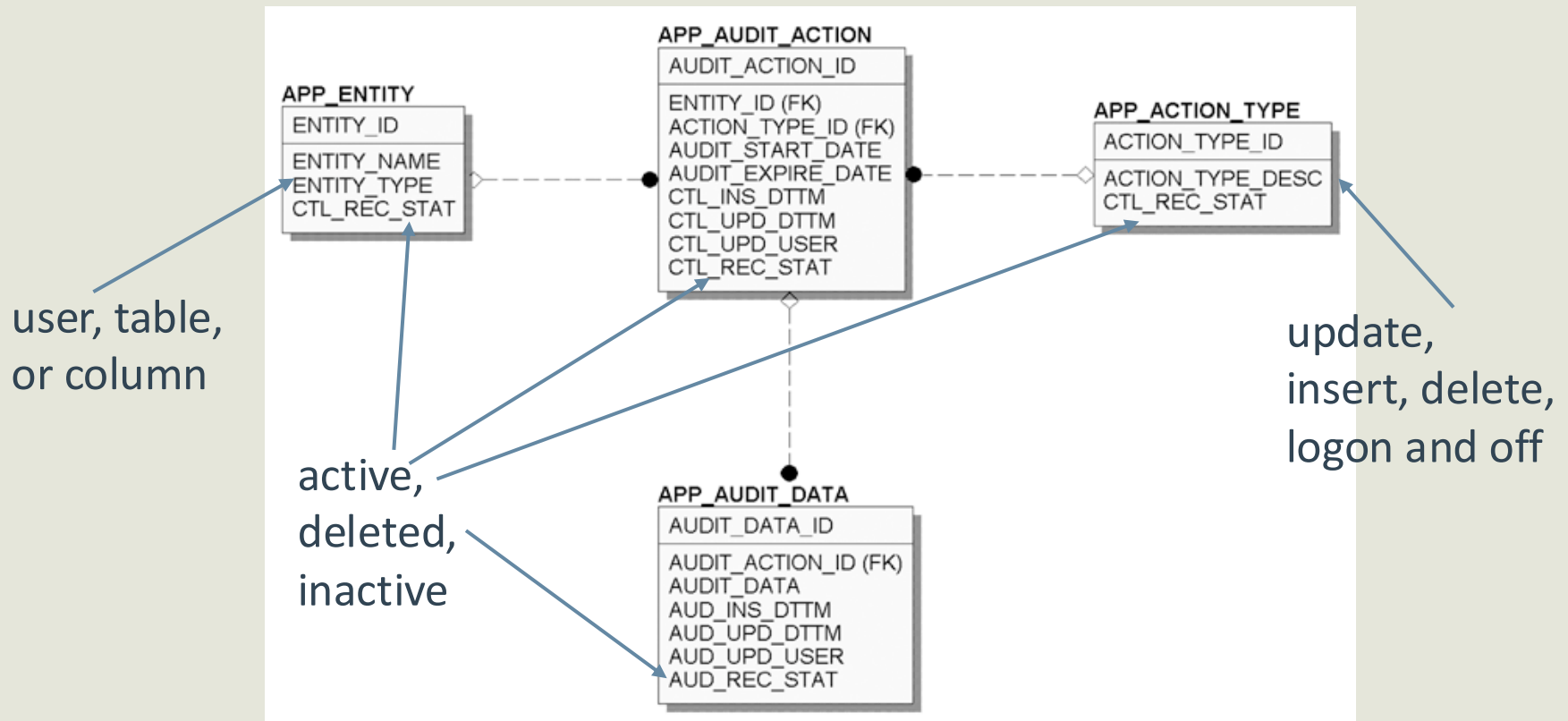
Auditing model 1



Auditing model 1

- Easy to understand and develop
- Registers audited entities in the audit model repository
- Chronologically tracks activities performed
- Entities: user, table, or column
- Activities: DML transaction (update, insert, delete) or logon and off times
- Status: active, deleted, inactive

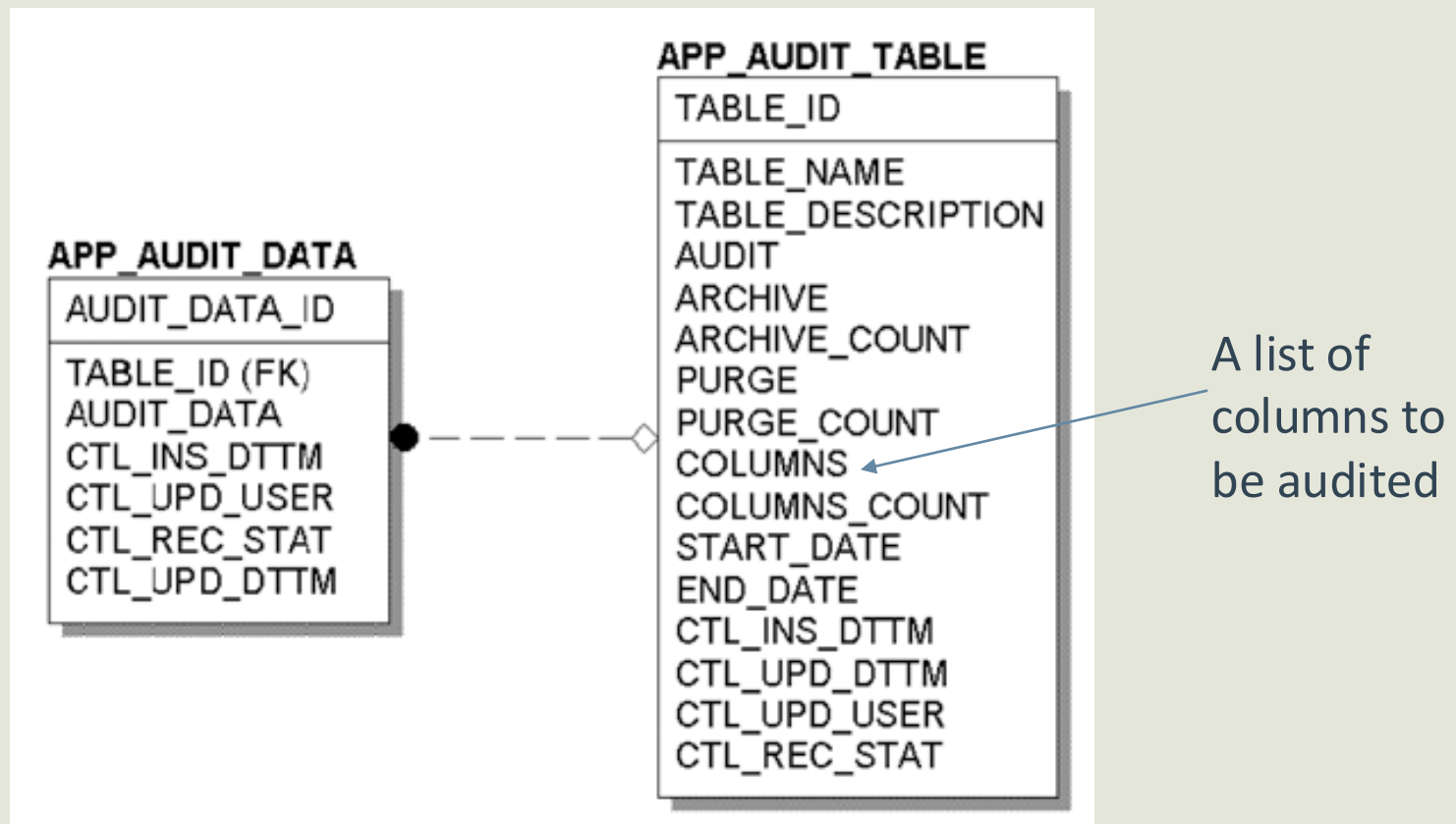
Auditing model 1



Auditing model 2

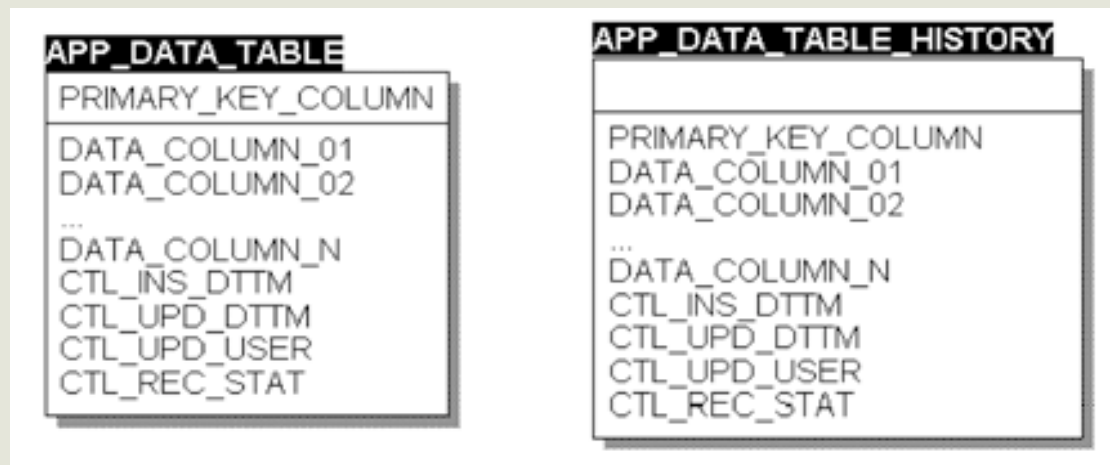
- Only stores the column value changes
- There is a purging and archiving mechanism
 - reduces the amount of data stored
- Does not register an action that was performed on the data
- Ideal for auditing a column or two of a table

Auditing model 2



Historical auditing model

- Used when a record of the whole row is required
- Typically used in most financial applications



Examples

- Example in MS SQL

```
CREATE SERVER AUDIT AuditDataAccess  
  TO FILE ( FILEPATH = 'C:\SQLAudit\' )  
  WHERE object_name = 'Employees' ;
```

```
ALTER SERVER AUDIT AuditDataAccess WITH (STATE = ON);
```

- Example in Oracle

```
CREATE AUDIT POLICY AuditDataAccess ACTIONS  
  DELETE on Employees,  
  INSERT on Employees,  
  UPDATE on Employees,  
  SELECT on Employees  
  CONTAINER = CURRENT;
```

```
AUDIT POLICY AuditDataAccess;
```

Conclusions

To make information systems compliant with the GDPR, multiple security measures should be considered:

- Attribute based access control (ABAC)
- Encryption (Always Encrypted/Transparent Data Encryption)
- (Dynamic) Data Masking
- Auditing

but

Are these security measures enough to protect privacy?